



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКИМ РАБОТАМ**

по дисциплине

«Проектирование клиент-серверных систем»

Выполнил студент группы ИКБО-01-22

Прокопчук Р.О.

Принял преподаватель кафедры ИиППО

Мельников Д.А.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва
2025

ОГЛАВЛЕНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №1	4
1.1 Описание системы	4
2 ПРАКТИЧЕСКАЯ РАБОТА №2	11
2.1 Use-Case диаграмма	11
2.2 Прецедент 1: создать сбор документов	11
3 ПРАКТИЧЕСКАЯ РАБОТА №3	15
3.1 Глоссарий.....	15
3.2 Подсферы для моделирования данных.....	16
3.3 Диаграмма классов UML.....	18
3.4 Диаграмма классов UML.....	18
3.5 Карта соответствий	21
Карта соответствий представлена в таблице 4.....	21
4 ПРАКТИЧЕСКАЯ РАБОТА №4	23
4.1 BPMN	23
4.2 Матрица RACI.....	24
5 ПРАКТИЧЕСКАЯ РАБОТА №5	26
5.1 Табличная спецификация API	26
5.2 Диаграммы последовательностей	34
5.3 Таблица надежности	36
5.4 Политика версионирования	37
5.5 Аутентификация/авторизация/безопасность.....	37
6 ПРАКТИЧЕСКАЯ РАБОТА №6	39
6.1 Карта доменных событий.....	39
7 ПРАКТИЧЕСКАЯ РАБОТА №7	40
7.1 Сценарий и допущения.....	40
7.2 Диаграмма последовательности	40
7.3 Диаграммы состояний	41
7.4 Модель репликации состояния.....	43
7.5 Окна лаг-компенсации и сглаживания	44

7.6 Таблица параметров обновления.....	45
7.7 Связь с профилем.....	45
7.8 Наблюдаемость.....	46
8 ПРАКТИЧЕСКАЯ РАБОТА №8	48
8.1 Эволюция контракта	48
8.2 ER-diff.....	49
8.3 Матрица совместимости.....	50
8.4 Правила Negotiation и Деградации	51
8.5 Политика Депрекейта	51
8.6 Миграция данных.....	52
8.7 Наблюдаемость.....	52
8.8 Наблюдаемость.....	53
9 ПРАКТИЧЕСКАЯ РАБОТА №9	55
9.1 Сценарии	55
9.2 Измеримые нефункциональные требования	55
9.3 Используемый паттерн и обоснование	56
10 ПРАКТИЧЕСКАЯ РАБОТА №10	57
10.1 Описание системы	57
10.2 C4-Context диаграмма.....	57
10.3 C4-Container диаграмма.....	57
10.4 C4-Component диаграмма.....	58
10.5 C4-Landscape диаграмма	59

1 ПРАКТИЧЕСКАЯ РАБОТА №1

1.1 Описание системы

Тема: Клиент-серверная система запроса документов по бухгалтерским операциям для клиентов B2B.

Цель: Система предназначена для автоматизации запроса, сбора и верификации документов, необходимых для подтверждения бухгалтерских операций клиентов B2B. Бизнес-ценность — снижение ручной работы бухгалтерии, ускорение обработки запросов клиентов, повышение качества данных для аналитики, централизованное хранение всех запрашиваемых документов, уменьшение числа касаний клиента при выполнении бизнес-процесса. Границы нашей ответственности включает в себя обеспечение процесса запроса документов у клиента, трекинг статусов сборов документов и интеграцию с внешними системами для передачи данных. Мы не отвечаем за дальнейший бухгалтерский процесс работы с документами и его влияние на клиентский опыт.

Акторы:

- **Бухгалтер** — Сотрудник, обслуживающий клиентов (B2B). Иницирует и обрабатывает запросы документов, утверждает/отклоняет документы, загружает документы самостоятельно при необходимости.
- **Клиент** — Конечный пользователь услуги (представитель компании-клиента). Получает запросы, загружает необходимые документы через Веб или мобильное приложение.
- **SRE-инженер** — следит за состоянием системы и очередей/процессов, исправляет ошибки, реагирует на инциденты.

Внешние системы:

- **Хранилище документов** — Сервис, отвечающий за хранение документов и их метаданных в компании.
- **Система верификации документов** — Сервис, отвечающий за валидацию формата присланных клиентом документов и определение их соответствия запрошенному типу.

- **Система нотификации** — Сервис, отвечающий за оповещение клиентов уведомлений.
- **Система клиентской авторизации SSO** — Сервис, отвечающий за авторизацию и ролевой доступ клиентов.
- **Система межсервисной авторизации** — Сервис, отвечающий за авторизацию серверных систем при выполнении запросов между собой.
- **Хранилище данных DWH** — Сервис, отвечающий за прием и хранение потоковых данных системы для дальнейшей аналитики.
- **Панель оркестратора процессов** — Сервис, отвечающий за отображение всех исторических и запущенных процессов сборов документов, возникших ошибок и инцидентов.

1.2 Stakeholder-map – карта заинтересованных сторон

Карта заинтересованных сторон представлена в таблице 1.

Таблица 1 – Карта заинтересованных сторон

Роль	Ожидания	Опасения	Метрика успеха
Владелец бизнеса	Повышение скорости предоставления услуг бухгалтерии. Снижение затрат на обслуживание клиентов за счет освобождения рабочего времени бухгалтеров.	Низкая окупаемость инвестиций. Затраты на доработку.	Время на обработку загруженных клиентом документов для 95% сборов документов ≤ 48 часов.
Бухгалтер	Удобный интерфейс для работы. Прозрачная система статусов запросов документов.	Получение не валидных документов. Сложность в обучении работе в новой системе.	95% ключевых операций (кроме создания сбора документов) выполняются ≤ 1 секунды. 95% операций по созданию сбора документов выполняются ≤ 30 секунд. 95% операций по передаче документов между системами ≤ 5 секунд.

Клиент	Простой процесс загрузки документов, уведомления и подтверждение получения	Потеря уведомлений и загруженных документов. Непонятный UI.	Конверсия на отправку требуемых документов $\geq 85\%$ в первые 72ч.
SRE-инженер	Возможность администрирования процессов, понятные логи и метрики.	Нестабильная работы системы.	Доступность услуги по запросу документов $\geq 98.5\%$. Доступность услуги по загрузке и скачиванию документов $\geq 99,9\%$
Аналитик данных	Полнота и качество данных в DWH.	Потеря событий. Неполные/несогласованные схемы данных.	Кол-во утерянных записей при поставке в DWH $\leq 0.1\%$.
Служба информационной безопасности	Соответствие политикам конфиденциальности и безопасности, шифрование, аудит доступов, минимизация прав.	Утечка персональных данных или документов. Загрузка опасных файлов.	100% всех сетевых операций требуют авторизации. 100% разрешенных для загрузки документов проходят валидацию.

Система запроса документов по бухгалтерским операциям включает веб-приложения для клиентов и бухгалтеров, а также мобильное приложение для клиентов. Ядро системы обрабатывает бизнес-логику, управляет обменом данными и файлами, взаимодействует с оркестратором процессов и внешними сервисами. Backend BPM координирует выполнение процессов сбора документов, а Backend бухгалтерии интегрирует функциональность сервиса в существующие бухгалтерские процессы.

Система использует корпоративный SSO для аутентификации, MSA для межсервисной авторизации, хранилище документов и сохранения файлов, сервис верификации для проверки формата файлов, систему нотификации для уведомлений и DWH для передачи данных в аналитику. Управление процессами осуществляется через административную панель оркестратора, доступную SRE-инженерам.

1.3 C4-Context диаграмма

C4-Context диаграмма представлена на рисунке 1.

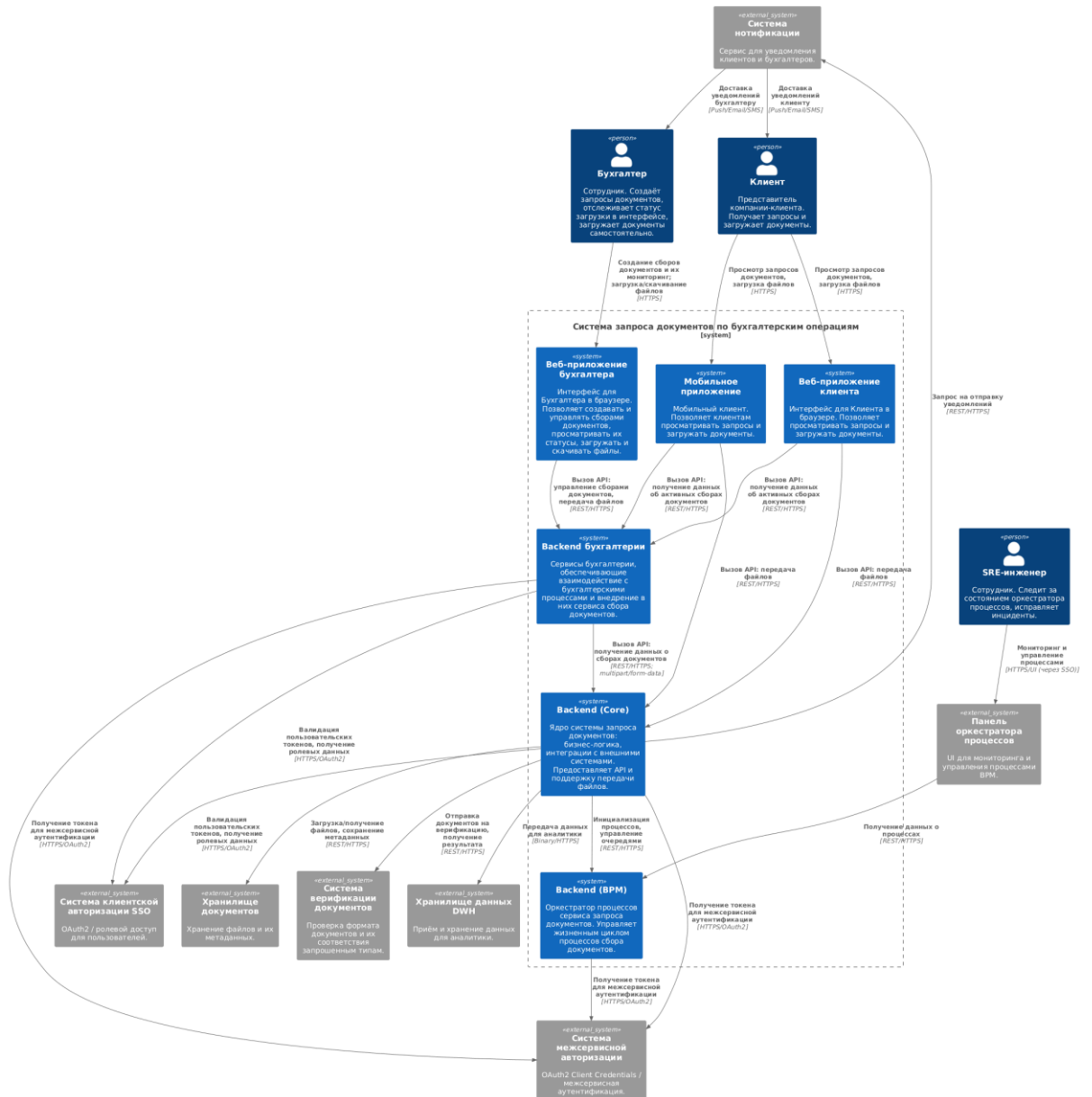


Рисунок 1 – C4-Context диаграмма

1.4 Utility Tree (NFR) – дерево нефункциональных требований

Дерево нефункциональных требований представлено на рисунке 2.



Рисунок 2 — Дерево нефункциональных требований

Связь нефункциональных требований с акторами и сценариями:

Performance & Latency

p95 ключевых операций ≤ 1 сек: Бухгалтер просматривает список активных сборов документов, обновляет статусы, скачивает файлы в процессе обработки 50+ запросов в день;

p95 создания сбора ≤ 30 сек: Бухгалтер инициирует новый сбор документов для клиента с выбором типов документов и отправкой уведомлений;

p95 загрузки документа ≤ 5 сек: Клиент загружает документы (до 10 МБ) через веб или мобильное приложение при нестабильном интернет-соединении.

Availability & Reliability

Доступность услуги по запросу документов $\geq 98.5\%$: Бухгалтер и Клиент критически зависят от системы в рабочее время (пн-пт 9:00-18:00) для выполнения бухгалтерских операций;

Доступность услуги по загрузке и скачиванию документов $\geq 99.9\%$:
Клиент загружает документы в срок, Бухгалтер получает файлы для дальнейшей обработки без задержек.

MTTR ≤ 30 мин: SRE-инженер обнаруживает инцидент через панель оркестратора процессов и восстанавливает работоспособность системы.

Security & Privacy

100% клиентских операций с авторизацией: Клиент и Бухгалтер проходят аутентификацию через SSO при каждом обращении к системе;

100% межсервисных операций с авторизацией: Все взаимодействия между Backend Core, Backend BPM, Backend бухгалтерии и внешними системами требуют токена MSA;

TLS 1.3+ шифрование: Все акторы защищены от перехвата данных при передаче документов и персональной информации.

Валидация при загрузке для 100% разрешенных типов документов: Клиент загружает только допустимые форматы файлов (PDF, JPG, PNG, ZIP); система верификации блокирует потенциально опасные файлы;

Валидация вложенных файлов в архивах по тем же правилам: Система верификации проверяет содержимое ZIP-архивов для предотвращения загрузки вредоносного ПО.

Observability & Monitoring

Логирование и трассировка 100% критичных операций: SRE-инженер отслеживает создание сборов, загрузку документов, верификацию через сквозной trace ID для расследования инцидентов;

Мониторинг 100% внешних интеграций: SRE-инженер видит статус подключений к хранилищу документов, системе верификации, DWH, системе нотификации в реальном времени.

Data Quality & Consistency

Кол-во утерянных записей при поставке в DWH $\leq 0.1\%$: Аналитик данных строит достоверные отчеты по срокам обработки документов и конверсии клиентов на основе полных данных.

Usability

Обработка документов после загрузки клиентом ≤ 48 ч для 95%:

Бухгалтер утверждает или отклоняет документы в течение 2 рабочих дней после загрузки, обеспечивая быстрый цикл обслуживания клиентов.

2 ПРАКТИЧЕСКАЯ РАБОТА №2

2.1 Use-Case диаграмма

Use-Case диаграмма представлена на рисунке 3.



Рисунок 3 – Use-Case диаграмма

2.2 Прецедент 1: создать сбор документов

Название: создать сбор документов

Primary Actor: Бухгалтер

Stakeholders/Interests: Клиент, Бухгалтер

Предусловия: Бухгалтер успешно авторизован в системе и находится на странице операции, по которой необходимо сделать запрос документов.

Триггер: Бухгалтер нажимает кнопку "Запросить документы".

Основной поток:

1. Диспетчер вводит данные по сбору (компания, контрагент, тип документа, период действия документа);
2. Система валидирует введенные данные;
3. Система создает сбор документов и сохраняет данные в БД;

4. Система отправляет клиенту нотификацию о необходимости загрузки документов.

Альтернативы/Исключения:

1. Такие документы уже запрошены в других сборах документов → система сообщает об ошибке и не дает продолжить действие.

Постусловия (успех): Сбор документов создан, клиенту отправлено уведомление.

Постусловия (неуспех): Сбор документов не создан, бухгалтер получил сообщение об ошибке.

Бизнес-правила: Все обязательные поля нужно заполнить.

NFR этого кейса: $p95 \leq 30$ с, доступность $\geq 98.5\%$, логирование 100%, авторизация 100%.

2.3 Прецедент 2: загрузить документы (на стороне клиента)

Название: загрузить документы

Primary Actor: Клиент

Stakeholders/Interests: Клиент, SRE-инженер, Бухгалтер.

Предусловия: Клиент авторизован в системе, перешел по уведомлению о необходимости загрузить документы.

Триггер: Клиент получает уведомление о необходимости загрузки документов.

Основной поток:

1. Клиент загружает файлы по все нужным документом;
2. Система валидирует файлы;
3. Система сохраняет файлы;
4. Система отправляет файлы на верификацию;
5. Статус сбора документов обновляется;
6. Файлы верифицируются;
7. Статус сбора документов обновляется.

Альтернативы/Исключения:

1. Загружены невалидные файлы → клиент получает сообщение об ошибке.

2. В процессе обработки сбора документов произошла ошибка → в панели оркестратора процесса отобразится инцидент, который увидит SRE-инженер.

Постусловия (успех): Статус сбора обновлен. Либо клиент получает повторный запрос (если документы не верифицированы), либо сбор документов переходит в терминальный статус, и документы отображаются в интерфейсе бухгалтера.

Постусловия (неуспех): Файлы не загружены → клиент видит ошибку; в процессе обработки сборов произошла ошибка → SRE исправляет проблему.

Бизнес-правила: можно загрузить только безопасные и поддерживаемые файлы.

NFR этого кейса: $p95 \leq 5$ с, доступность $\geq 99.9\%$, логирование 100%, авторизация 100%, валидация 100%.

2.4 Прецедент 3: посмотреть статус сбора документов

Название: посмотреть статус сбора документов

Primary Actor: Бухгалтер

Stakeholders/Interests: Бухгалтер

Предусловия: Бухгалтер авторизован в системе и находится на странице операции.

Триггер: Бухгалтер решил проверить статус обработки документов по операции.

Основной поток:

1. Бухгалтер открывает интересующий его сбор документов;
2. Бухгалтеру отображает вся информация о сборе.

Альтернативы/Исключения:

1. Ошибка при получении сбора → система сообщает об ошибке.

Постусловия (успех): Бухгалтер видит текущий статус сбора документов

Постусловия (неуспех): Сбор не отображается, ошибка отображается бухгалтеру.

Бизнес-правила: Сбор документов должен существовать.

NFR этого кейса: $p95 \leq 1$ с, доступность $\geq 98.5\%$, авторизация 100%.

2.5 Матрица трассировки

Матрица трассировки представлена в таблице 2.

Таблица 2 – Карта заинтересованных сторон

Актор	Use Case	Связанные NFR	Примечание
Бухгалтер	Создать сбор документов	$p95 \leq 30$ сек, доступность $\geq 98.5\%$, логирование 100%, авторизация 100%	Критичная операция для начала процесса, но нет влияния на клиента
Бухгалтер	Посмотреть статус сбора документов	$p95 \leq 1$ сек, доступность $\geq 98.5\%$, авторизация 100%	Нет прямого влияния на клиента, но важно, чтобы бухгалтер мог своевременно выполнять работу.
Бухгалтер	Загрузить документы	$p95 \leq 5$ сек, доступность $\geq 99.9\%$, валидация 100%, TLS 1.3+	Требования привязаны к клиентским.
Бухгалтер	Скачать документы	$p95 \leq 5$ сек, доступность $\geq 99.9\%$, авторизация 100%, TLS 1.3+	Требования привязаны к клиентским.
Клиент	Посмотреть уведомление	$p95 \leq 1$ сек, доступность $\geq 99.9\%$, авторизация 100%	Клиент должен иметь возможность быстро получать информацию о статусе заявки для повышения удовлетворенности.
Клиент	Загрузить документы	$p95 \leq 5$ сек, доступность $\geq 99.9\%$, валидация 100%, TLS 1.3+	Критическая операция для всего процесса. Важна постоянная доступность услуги и ее безопасность. В случае неудачи случается прямое воздействие на клиента.
Клиент	Скачать документы	$p95 \leq 5$ сек, доступность $\geq 99.9\%$, авторизация 100%, TLS 1.3+	Важная операция для клиентского опыта. Важна постоянная доступность услуги и ее безопасность
SRE-инженер	Посмотреть инциденты по процессам сбора	доступность $\geq 99.5\%$	Критически важно вовремя исправлять инциденты.

3 ПРАКТИЧЕСКАЯ РАБОТА №3

3.1 Глоссарий

1. Сбор документов (Collection) — единица работы системы, объединяющая запрос нескольких документов у клиента для конкретной бухгалтерской операции.
2. Документ коллекции (CollectionDocument) — запрашиваемый документ в рамках сбора с указанием типа, периода и комментариев бухгалтера.
3. Документ (Document) — загруженный документ с метаданными (тип, компания, контакт), содержащий файлы и связанные задачи.
4. Файл (File) — физический файл документа, хранящийся в хранилище документов с метаданными о загрузке.
5. Задача на загрузку (UploadTask) — процесс отслеживания загрузки документа клиентом или бухгалтером.
6. Задача на верификацию (VerifyTask) — процесс проверки документа системой верификации на соответствие требованиям и шаблону.
7. Информация DocStorage (DocStorageDocumentInfo) — агрегированные данные о документе (все загруженные файлы) из внешнего хранилища документов.
8. Тип документа (DocumentType) — категория документа (счет-фактура, акт выполненных работ, договор).
9. Статус (Status) — этап жизненного цикла сущности (NEW, IN_PROGRESS, UPLOADED, VERIFIED, REJECTED).
10. Компания (Company) — клиент B2B, для которого запрашиваются документы (идентифицируется через company_id).
11. Контакт (Contact) — представитель компании-клиента, который загружает документы (идентифицируется через contact_id).
12. Шаблон верификации (VerificationTemplate) — набор правил проверки документа (формат, содержимое, обязательные поля).

13. Стратегия проталкивания (PushingStrategy) — способ доставки уведомлений клиенту (PUSH, EMAIL, SMS).

14. Период (Period) — временной интервал, к которому относится документ (начало и конец периода).

15. DocStorage ID — уникальный идентификатор файла во внешнем хранилище документов.

3.2 Подсферы для моделирования данных

Профиль (Управление участниками процесса)

Ключевые сценарии из Use Case:

1. Аутентификация пользователя: Система проверяет учетные данные бухгалтера/клиента через SSO.

2. Идентификация компании и контакта: Система связывает загруженные документы с конкретными компаниями и их представителями.

In Scope:

- Хранение идентификаторов компаний (company_id) и контактов (contact_id);
- Связь документов и сборов с компаниями;
- Отслеживание, кто загрузил файлы.

Out of Scope:

- Механизм SSO — рассматривается как внешняя система;
- Управление ролями и правами доступа — делегируется SSO;
- Детальная информация о пользователях — хранится в CRM.

Инвентарь (Управление сборами и документами)

Ключевые сценарии из Use Case:

1. Создать сбор документов: Бухгалтер инициирует запрос нескольких документов у клиента.

2. Посмотреть статус сбора документов: Бухгалтер проверяет, какие документы загружены и верифицированы.

In Scope:

- Сущности Collection, CollectionDocument, Document;

- Связь сбора с набором запрашиваемых документов (1:N);
- Связь документа с загруженными файлами (1:N);
- Статусная модель для сборов и документов;
- Периоды действия документов.

Out of Scope:

- Механизм дедупликации документов и разнообразная метайнформация;

- Механизмы кастомизации нотификаций;
- Комментарии бухгалтера к сборам документов;
- Механизм авто-завершения сбора;
- Отклонение сбора документов.

Экономика (Обработка файлов).

Ключевые сценарии:

- Скачать файлы: Клиент или бухгалтер скачивает файлы;
- Загрузить документы: Клиент или бухгалтер загружает файлы для сбора.
- Верификация документа: Система отправляет документ в систему верификации и отслеживает результат;
- Интеграция с хранилищем документов: Система агрегирует информацию о документах из внешнего хранилища;

In Scope:

- Сущности UploadTask, VerifyTask, DocStorageDocumentInfo, File;
- Связь задач с документом;
- Хранение идентификаторов файлов в хранилище документов;
- Статусы загрузки и верификации;

Out of Scope:

- Сложная логика кастомизируемого flow сбора при помощи шаблонов верификации;
- Типизация документов в системе верификации документов;

- Сложная логика обработки метаданных документов в хранилище;
- Дедупликация задач на загрузку и верификацию.

3.3 Диаграмма классов UML

Диаграмма классов представлена на рисунке 4.

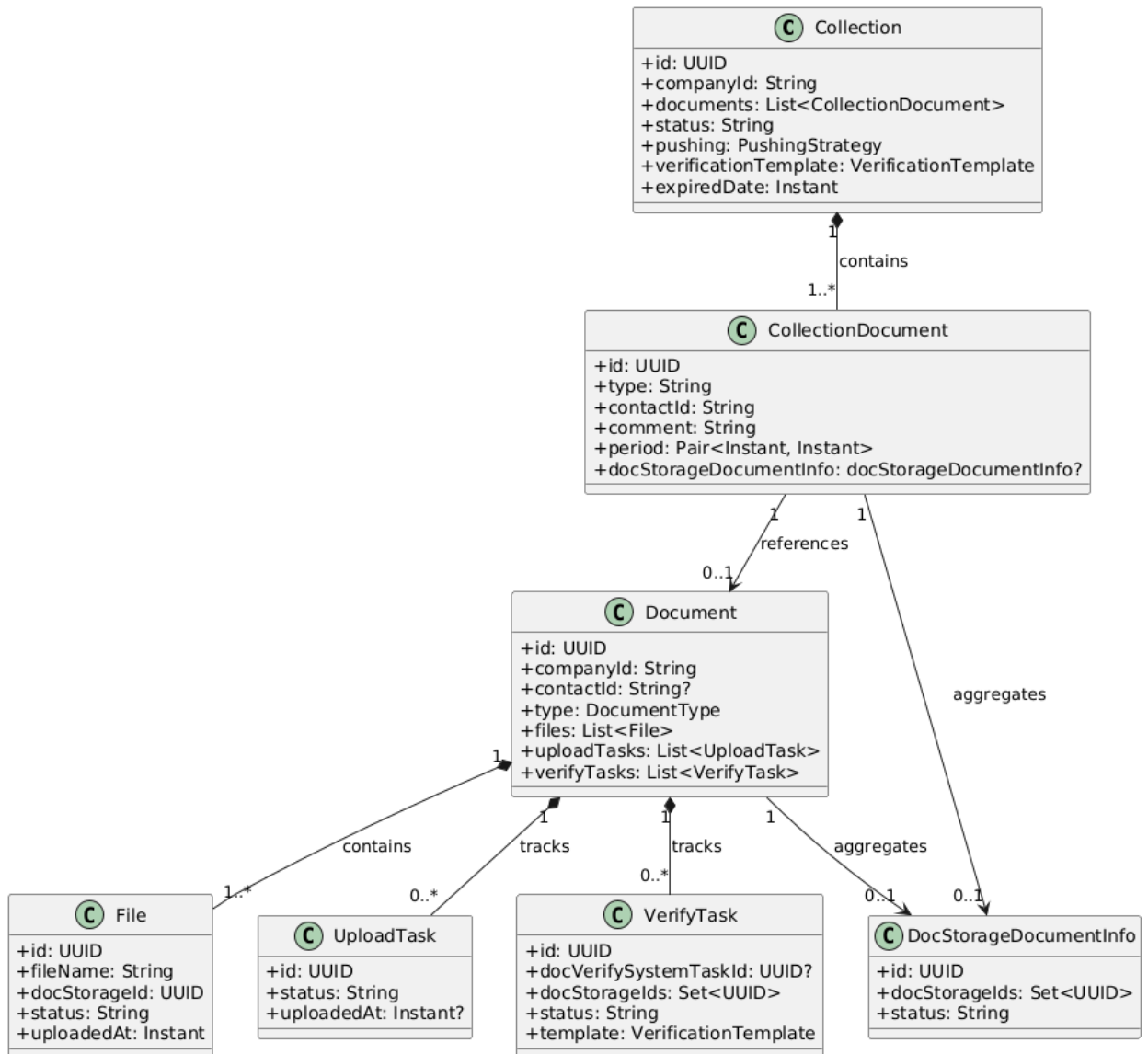


Рисунок 4 – Диаграмма классов

3.4 Диаграмма классов UML

ER-диаграмма представлена на рисунке 5.

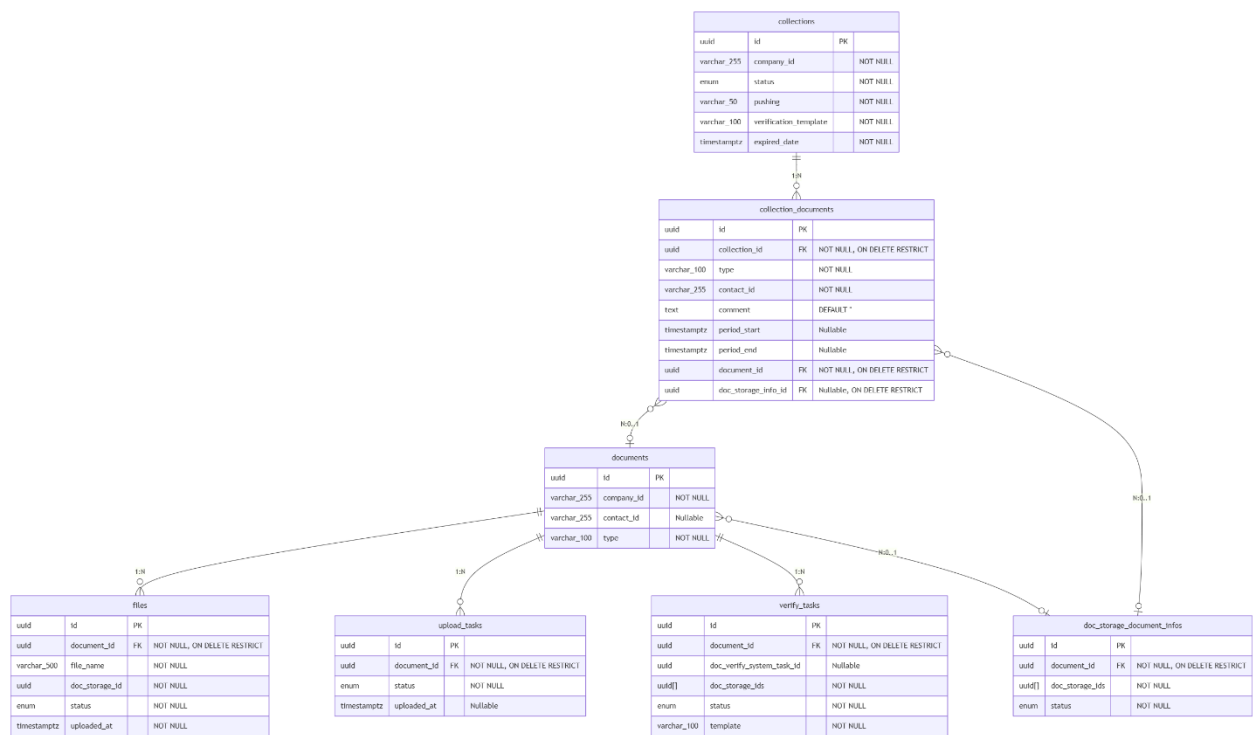


Рисунок 5 – ER-диаграмма

Инженерные решения:

- PK для всех сущностей: UUID — для обеспечения уникальности в распределенной системе и невозможности перебора;
- Время: TIMESTAMPTZ (UTC);
- Строковые данные с ограничениями длины (varchar);
- ON DELETE RESTRICT для всех FK — запрет каскадного удаления для предотвращения потери данных.

Описание полей сущностей представлено в таблице 3.

Таблица 3 – Описание полей сущностей

Сущность	Атрибут	Тип	Обязательно	Ограничение/Комментарий
collections	id	UUID	Да	PK
	company_id	VARCHAR(255)	Да	Идентификатор компании-клиента
	status	VARCHAR(50)	Да	ENUM
	pushing	VARCHAR(50)	Да	ENUM
	verification_template	VARCHAR(100)	Да	Код шаблона верификации для всех документов сбора

	expired_date	TIMESTAM PTZ	Да	Дедлайн загрузки документов
collection_documents				
	id	UUID	Да	PK
	collection_id	UUID	Да	FK → collections(id), ON DELETE RESTRICT
	type	VARCHAR(100)	Да	Тип запрашиваемого документа
	contact_id	VARCHAR(255)	Да	Идентификатор контакта (представителя компании)
	comment	TEXT	Да	DEFAULT "", комментарий бухгалтера
	period_start	TIMESTAM PTZ	Нет	Начало периода действия документа
	period_end	TIMESTAM PTZ	Нет	Конец периода действия документа
	document_id	UUID	Да	FK → documents(id), ON DELETE RESTRICT
	doc_storage_info_id	UUID	Нет	FK → doc_storage_document_i nfos(id), ON DELETE RESTRICT
documents				
	id	UUID	Да	PK
	company_id	VARCHAR(255)	Да	Идентификатор компании
	contact_id	VARCHAR(255)	Нет	Идентификатор контакта, загрузившего документ
	type	VARCHAR(100)	Да	Тип документа
files				
	id	UUID	Да	PK
	document_id	UUID	Да	FK → documents(id), ON DELETE RESTRICT
	file_name	VARCHAR(500)	Да	Исходное имя файла (до 500 символов)
	doc_storage_id	UUID	Да	UUID файла в хранилище документов
	status	VARCHAR(50)	Да	ENUM

	uploaded_at	TIMESTAM PTZ	Да	Время загрузки файла
upload_tasks				
	id	UUID	Да	PK
	document_id	UUID	Да	FK → documents(id), ON DELETE RESTRICT
	status	VARCHAR(50)	Да	CHECK (status IN ('NEW', 'IN_PROGRESS', 'UPLOADED', 'FAILED'))
	uploaded_at	TIMESTAM PTZ	Нет	Время завершения загрузки
verify_tasks				
	id	UUID	Да	PK
	document_id	UUID	Да	FK → documents(id), ON DELETE RESTRICT
	doc_verify_system_ task_id	UUID	Нет	UUID задачи в системе верификации
	doc_storage_ids	UUID[]	Да	Массив UUID файлов для верификации
	status	VARCHAR(50)	Да	ENUM
	template	VARCHAR(100)	Да	Код шаблона верификации
doc_storage_ document_infos				
	id	UUID	Да	PK
	document_id	UUID	Да	FK → documents(id), ON DELETE RESTRICT
	doc_storage_ids	UUID[]	Да	Массив UUID файлов в хранилище документов
	status	VARCHAR(50)	Да	ENUM

3.5 Карта соответствий

Карта соответствий представлена в таблице 4.

Таблица 4 – Карта соответствий

Домен (Класс)	Таблица	Комментарий
Collection	collections	Содержит метаданные сбора: стратегию уведомлений, общий шаблон верификации, дедлайн.

CollectionDocument	collection_documents	Хранит период действия документа и комментариев бухгалтера.
Document	documents	Может быть связан с несколькими CollectionDocument (один документ в нескольких сборах).
File	files	Хранит UUID файла и мету в хранилище документов и время загрузки.
UploadTask	upload_tasks	Упрощенная модель отслеживания загрузки.
VerifyTask	verify_tasks	Хранит массив UUID файлов для верификации и шаблон проверки.
DocStorageDocumentInfo	doc_storage_document_infos	Консолидирует информацию о файлах для собранного документа.

4 ПРАКТИЧЕСКАЯ РАБОТА №4

4.1 BPMN

BPMN-диаграмма представлена на рисунке 6.

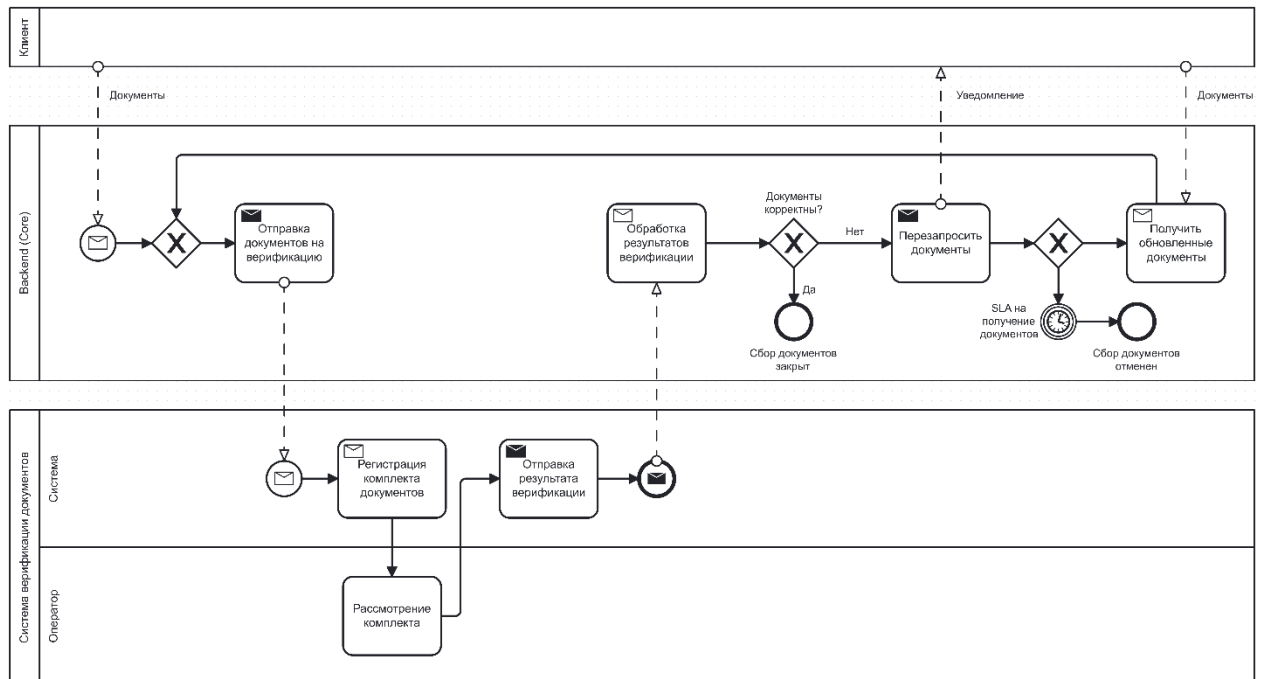


Рисунок 6 – ER-диаграмма

Правила входов/выходов представлена в таблице 5.

Таблица 5 – Правила входов/выходов этапов

Этап	Вход (условия / данные)	Действие	Выход (состояние / событие)	SLA	Ответствен ный
Поступле ние запроса	Документы по сбору	Система сбора документов отправляет документы на верификацию в систему верификации	Статус сбора = «отправлен на верификац ию»	≤ 2 мин	Система сбора документов
Регистрац ия комплекта документ ов	Комплект документо в для верификац ии	Система сохраняет запрос и направляет его оператору	Статус комплекта = «на проверке»	≤ 10 мин	Система верификаци и документов

Рассмотрение комплекта	Комплект документов	Оператор валидирует документы, заполняет поля и отмечает ошибки	Статус комплекта = «выполнен»	≤ 60 мин	Оператор системы верификации документов
Отправка результата верификации	Выполненный комплект	Система сохраняет данные выполненного комплекта и отправляет результат	Статус комплекта = «результат отправлен»	≤ 2 мин	Система верификации документов
Обработка результатов верификации	Результат верификации документов	Система сбора документов проверяет, соответствует ли комплект условиям	Статус сбора = «завершен» или «требуется повторного запроса документов»	≤ 1 мин	Система сбора документов
Перезапрос документов	Информация о неверифицированных документах	Система отправляет пользователю уведомление о необходимости повторной загрузки	Уведомление отправлено	≤ 2 мин	Система сбора документов
Получение обновленных документов	Обновленные документы	Система получает документы и передает их дальше по процессу	Статус сбора = «документы загружены»	≤ сконфигурированный на старте сбора SLA	Система сбора документов

4.2 Матрица RACI

Матрица RACI представлена в таблице 6.

Таблица 6 – Матрица RACI

Этап	Пользователь	Система сбора документов	Система верификации документов	Оператор системы верификации
1. Поступление запроса (отправка документов)	R/A	I	—	—
2. Отправка документов на верификацию	C	R/A	I	—
3. Регистрация комплекта документов	—	C	R/A	I
4. Рассмотрение комплекта	—	—	C	R/A
5. Отправка результата верификации	—	I	R/A	C
6. Обработка результатов верификации	—	R/A	I	—
7. Перезапрос документов (уведомление пользователю)	I	R/A	—	—
8. Повторная загрузка документов пользователем	R/A	I	—	—

5 ПРАКТИЧЕСКАЯ РАБОТА №5

5.1 Табличная спецификация API

Спецификация API разрабатываемой системы представлена в таблице 6.

Таблица 6 – Все эндпоинты системы

Resource	Operation	Method	Path	Auth	Description
collections	Create collection (идемпотентно)	POST	/v1/collections	Bearer	Создать новый сбор документов (если уже создавался с тем же Idempotency-Key — вернуть прежний ответ).
collections	Get collection by id	GET	/v1/collections/{collection_id}	Bearer	Получить сбор по id.
collections	List collections (фильтры)	GET	/v1/collections	Bearer	Получение сбора по фильтрам: id, company_id, contact_id, counterparty_id, type, period_start, period_end, status, page, limit, sort.
collections	Cancel collection	POST	/v1/collections/{collection_id}/cancel	Bearer	Отмена (soft) сбора — перевод в статус CANCELLED.
collection_documents	List requested documents for client	GET	/v1/collections/{collection_id}/documents	Bearer	Список документов, которые нужно загрузить по сбору с метаданными
documents	Upload file for document	POST	/v1/documents/{document_id}/files	Bearer	Загружает файл (multipart/form-data). Создаёт ресурс File и возвращает doc_storage_id.
files	Download file	GET	/v1/files/{file_id}/download	Bearer	Возвращает перенаправление/Si

					gned URL для скачивания.
files	Delete file	DELETE	/v1/files/{file_id}	Bearer	Удаление файла (soft), проверка прав.
documents	Submit uploaded files (submit button)	POST	/v1/collections/{collection_id}/submit	Bearer	Отправка всех загруженных файлов на верификацию. Идempотентна в пределах одного collection_id (если уже отправлено — вернуть статус).

5.1.1 POST /v1/collections – создание сбора документов

Спецификация данного эндпоинта представлена в таблицах 7-9 и в листингах 1-2.

Таблица 7 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJhbGci...
Idempotency-Key	R	UUID для идемпотентности	0d9aef2e-3c1b-4d7b-9f77-6c1fla3a0b0e

Таблица 8 – Спецификация тела запроса

Field	Type	Req	Rules / Enum	Example
company_id	uuid	R	FK → Company	"c_123"
counterparty_id	uuid	O	FK → Counterparty	"cp_555"
contact_id	uuid	O	FK → Contact	"u_321"
type	string	R	Enum: INVOICE_COLLECTION, CONTRACT_COLLECTION	"INVOICE_COLLECTION"
documents	array[object]	R	minItems ≥ 1	—
documents[].type	string	R	Enum: INVOICE, ACT, CONTRACT	"INVOICE"
documents[].period_start	string	O	RFC3339/UTC	"2025-01-01T00:00:00Z"

documents[].period_end	string	O	RFC3339/UTC	"2025-01-31T23:59:59Z"
documents[].required	boolean	R	true/false	true
pushing	string	O	Enum: EMAIL, SMS, PUSH	"EMAIL"
expired_date	string	O	RFC3339/UTC	"2025-10-01T00:00:00Z"

Листинг 1 – Пример запроса

```
{
  "company_id": "company_123",
  "counterparty_id": "cp_789",
  "contact_id": "contact_456",
  "type": "INVOICE_COLLECTION",
  "documents": [
    {
      "type": "INVOICE",
      "period_start": "2025-01-01T00:00:00Z",
      "period_end": "2025-01-31T23:59:59Z",
      "required": true
    }
  ],
  "pushing": "EMAIL",
  "expired_date": "2025-10-01T00:00:00Z"
}
```

Таблица 9 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
id	uuid	R	PK	"5d0e99c1-f0a9-4e2a-9cc7-ef7c7a7df92a"
company_id	uuid	R	FK → Company	"company_123"
type	string	R	Enum	"INVOICE_COLLECTION"
status	string	R	Enum: NEW, IN_PROGRESS, CANCELLED	"NEW"
created_at	string	R	RFC3339/UTC	"2025-09-01T12:00:00Z"

Код ответа – 201.

Листинг 2 – Пример ответа

```
{
  "id": "5d0e99c1-f0a9-4e2a-9cc7-ef7c7a7df92a",
  "company_id": "company_123",
```

```

"type":"INVOICE_COLLECTION",
"status":"NEW",
"created_at":"2025-09-01T12:00:00Z"
}

```

5.1.2 GET /v1/collections – получение списка сборов

Спецификация данного эндпоинта представлена в таблицах 10-12 и в листинге 3.

Таблица 10 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...

Таблица 11 – Спецификация тела query-параметров

Param	Type	Req	Rules / Enum	Example
id	uuid	O	фильтр по id	"5d0e..."
company_id	uuid	O	FK	"company_123"
counterparty_id	uuid	O	FK	"cp_456"
type	string	O	Enum	"INVOICE_COLLECTION"
status	string	O	Enum	"NEW"
period_start	string	O	RFC3339/UTC	"2025-01-01T00:00:00Z"
period_end	string	O	RFC3339/UTC	"2025-01-31T23:59:59Z"
page	int	O	≥ 1	1
limit	int	O	≤ 100	20
sort	string	O	`created_at:asc` `desc`	

Таблица 12 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
items	array[object]	R	список сборов	–
items[].id	uuid	R	PK	"5d0e..."
items[].company_id	uuid	R	FK	"company_123"
items[].type	string	R	Enum	"INVOICE_COLLECTION"
items[].status	string	R	Enum	"NEW"
items[].created_at	string	R	RFC3339/UTC	"2025-09-01T12:00:00Z"
page	int	R	≥ 1	1
limit	int	R	≤ 100	20

total	int	R	≥ 0	42
-------	-----	---	----------	----

Код ответа – 200.

Листинг 3 – Пример ответа

```
{
  "items":[
    {"id":"5d0e99c1-f0a9-4e2a-9cc7-ef7c7a7df92a", "company_id":"company_123", "type":"INVOICE_COLLECTION", "status":"NEW", "created_at":"2025-09-01T12:00:00Z"}
  ],
  "page":1,
  "limit":20,
  "total":1
}
```

5.1.3 GET /v1/collections/{id} – получение сбора по ID

Спецификация данного эндпоинта представлена в таблицах 13-14 и в листинге 4.

Таблица 13 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...

Таблица 14 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
id	uuid	R	PK	"5d0..."
company_id	uuid	R	FK	"company_123"
contact_id	uuid	O	FK	"u_1"
type	string	R	Enum	"INVOICE_COLLECTION"
status	string	R	Enum: NEW, IN_PROGRESS, UPLOADED, CANCELLED	"IN_PROGRESS"
documents	array[object]	R	список документов	–
documents[].id	uuid	R	PK	"doc_1"
documents[].type	string	R	Enum	"INVOICE"

documents[].period_start	string	O	RFC3339/UTC	"2025-01-01T00:00:00Z"
documents[].period_end	string	O	RFC3339/UTC	"2025-01-31T23:59:59Z"
documents[].required	boolean	R	true/false	true
documents[].status	string	R	Enum: NEW, UPLOADED, VERIFIED, REJECTED	"UPLOADED"

Код ответа – 200.

Листинг 4 – Пример ответа

```
{
  "id": "5d0e99c1-f0a9-4e2a-9cc7-ef7c7a7df92a",
  "company_id": "company_123",
  "type": "INVOICE_COLLECTION",
  "status": "IN_PROGRESS",
  "documents": [
    {
      "id": "doc_1",
      "type": "INVOICE",
      "period_start": "2025-01-01T00:00:00Z",
      "period_end": "2025-01-31T23:59:59Z",
      "required": true,
      "status": "UPLOADED"
    }
  ]
}
```

5.1.4 POST /v1/collections/{id}/cancel – отмена сбора

Спецификация данного эндпоинта представлена в таблицах 15-16.

Таблица 15 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...
If-Match	O	ETag для конкурентного контроля	"v1_23a5"

Таблица 16 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
id	uuid	R	PK	"5d0..."
status	string	R	Enum: CANCELLED	"CANCELLED"
updated_at	string	R	RFC3339/UTC	"2025-09-02T14:00:00Z"

Код ответа – 201.

5.1.5 GET /v1/collections/{id}/documents – список документов клиента

Спецификация данного эндпоинта представлена в таблицах 17-18.

Таблица 17 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен клиента	Bearer eyJ...

Таблица 18 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
collection_id	uuid	R	FK	"col_123"
items	array[object]	R	список документов	–
items[].id	uuid	R	PK	"doc_1"
items[].type	string	R	Enum	"INVOICE"
items[].required	boolean	R	true/false	true
items[].status	string	R	Enum	"UPLOADED"
items[].uploaded_files	array[object]	O	—	–
items[].uploaded_files[].file_id	uuid	R	FK	"f_1"
items[].uploaded_files[].file_name	string	R	—	"invoice.pdf"
items[].uploaded_files[].size	int	R	bytes	23122

Код ответа – 200.

5.1.6 POST /v1/documents/{id}/files – загрузка файла

Спецификация данного эндпоинта представлена в таблицах 19-21.

Таблица 19 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...
Idempotency-Key	R	UUID для идемпотентности	"8ab3..."

Таблица 20 – Спецификация формы запроса

Field	Type	Req	Rules / Enum	Example
file	binary	R	MIME: pdf, png, jpg ≤50MB	(файл)
uploader_id	uuid	R	FK → Contact	"u_456"

Таблица 21 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
file_id	uuid	R	PK	"f_1"

document_id	uuid	R	FK	"d_1"
file_name	string	R	–	"invoice2025.pdf"
size	int	R	bytes	34215
Status	string	R	Enum: UPLOADED	"UPLOADED"
uploaded_at	string	R	RFC3339/UTC	"2025-09-10T12:00:00Z"

Код ответа – 201.

5.1.7 POST /v1/collections/{id}/submit – отправка файлов (submit)

Спецификация данного эндпоинта представлена в таблицах 22-23.

Таблица 22 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...
Idempotency-Key	R	UUID	"submit-abc"

Таблица 23 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
collection_id	uuid	R	FK	"col_123"
verify_task_id	uuid	R	FK → VerifyTask	"vt_456"
status	string	R	Enum: SENT_FOR_VERIFICATION	"SENT_FOR_VERIFICATION"
submitted_at	string	R	RFC3339/UTC	"2025-09-10T12:00:00Z"

Код ответа – 202.

5.1.8 GET /v1/files/{file_id}/download – скачивание файла

Спецификация данного эндпоинта представлена в таблицах 24-25.

Таблица 24 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...

Таблица 25 – Спецификация тела ответа

Field	Type	Req	Rules / Enum	Example
location	string	R	Signed URL, TTL ≤60s	"https://storage.example.com/file/123?sig=abc"

Код ответа – 302.

Пример ответа – <https://storage.example.com/file/123?sig=abc>.

5.1.9 DELETE /v1/files/{file_id} – удаление файла

Спецификация данного эндпоинта представлена в таблицах 26.

Таблица 26 – Спецификация заголовков

Header	Req	Description	Example
Authorization	R	Bearer JWT токен	Bearer eyJ...

Код ответа – 204.

5.1.10 Норматив по кодам и ошибкам

Норматив по кодам и ошибкам представлен в таблице 27. Пример общего формата ошибок – на листинге 5.

Таблица 6 – Все эндпоинты системы

Code	When	Body
200	успешное чтение	объект/массив
201	создан ресурс	Location + объект
202	принято в обработку	{status:"accepted"}
204	удалено / нет тела	–
400	неверный запрос	{code:"invalid", field:"...", message:"..."}
401	нет/некорректен токен	{code:"unauthorized"}
403	нет прав	{code:"forbidden"}
404	не найдено	{code:"not_found"}
409	конфликт версий / идемпотентности	{code:"conflict"}
422	ошибка доменной валидации	{code:"invalid_field", field:"...", message:"..."}
429	превышен лимит	Retry-After header
5xx	сбой сервера	{code:"server_error", traceId:"..."}

Листинг 5 – Единый формат ошибки

```
{
  "code": "string",
  "message": "string",
  "field": "string|null",
  "traceId": "uuid"
}
```

5.2 Диаграммы последовательностей

Диаграммы последовательностей представлены на рисунках 7-9.

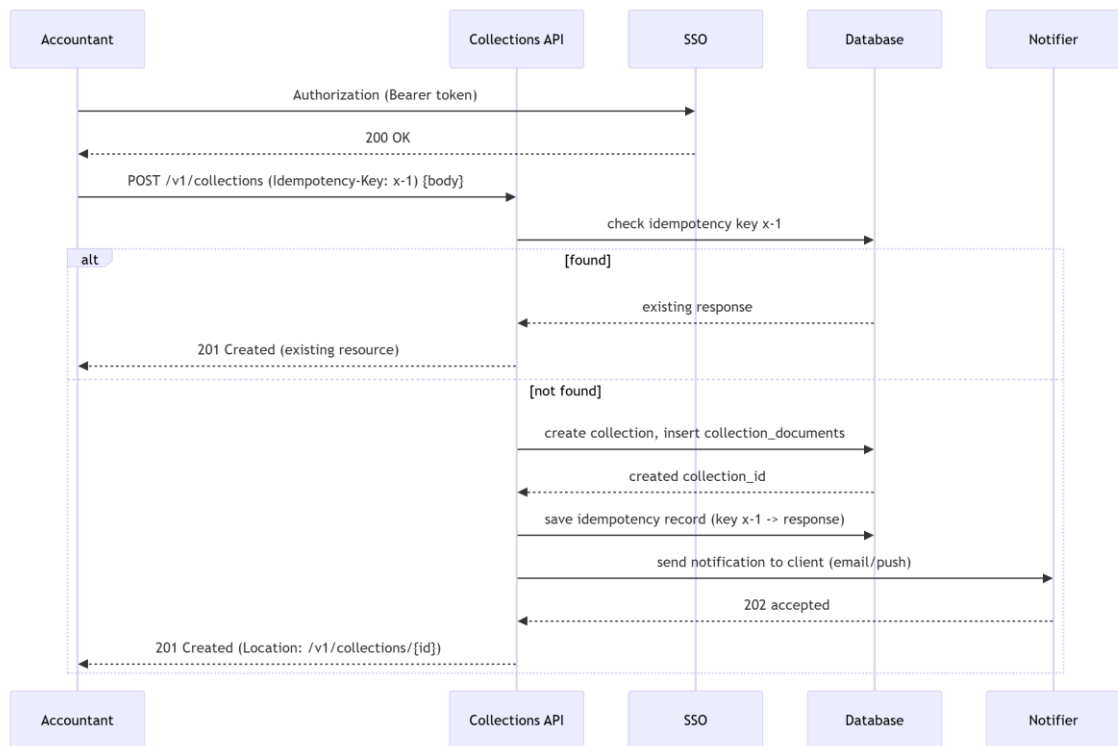


Рисунок 7 – Последовательность создания сбора документов

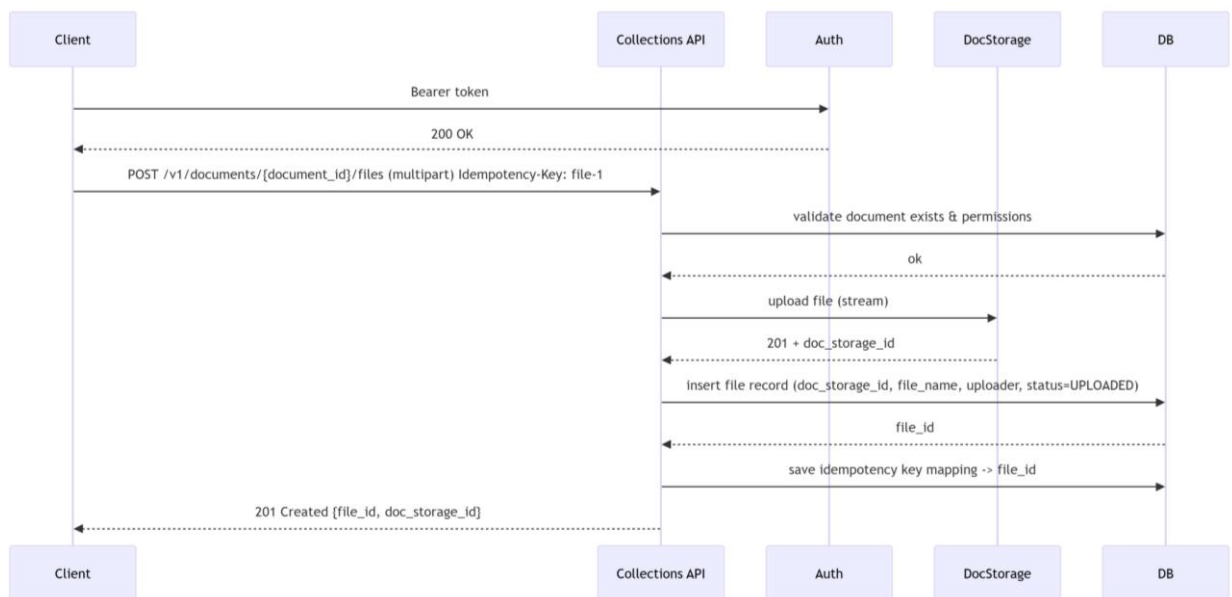


Рисунок 8 – Последовательность загрузки файлов

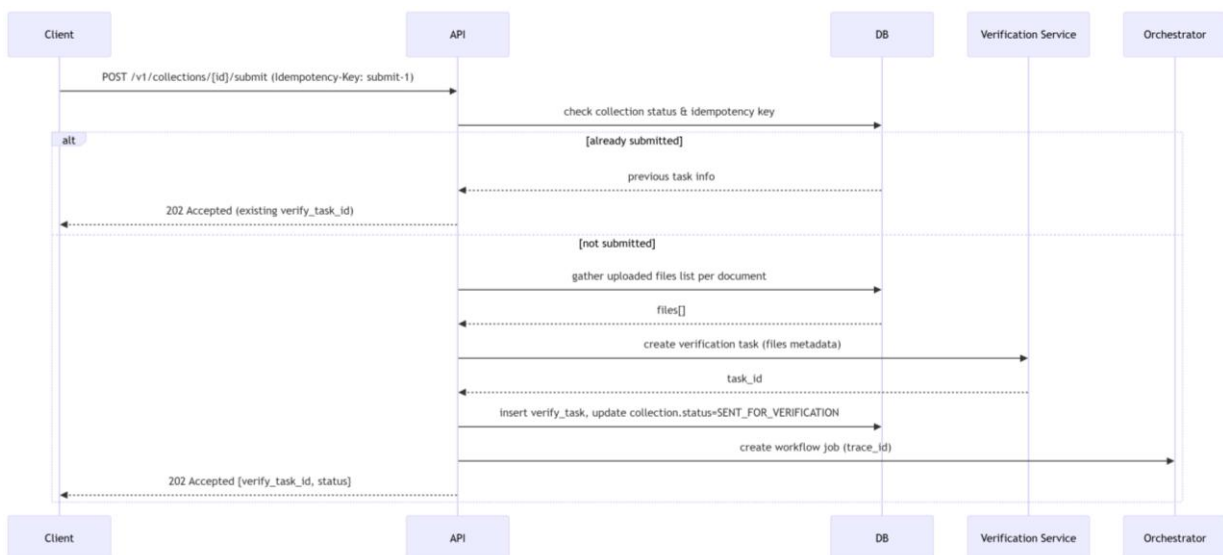


Рисунок 9 – Последовательность подтверждения загрузки файлов

5.3 Таблица надежности

Таблица надежности представлена в таблице 28.

Таблица 28 – Таблица надежности

Operation	Safe?	Идемпо- тентна?	Key/Scope	Retry when	Backoff	Client timeout	Server timeout
GET /v1/collections	yes	yes	—	5xx	none/short	2s	1.5s
POST /v1/collections	no	yes	Idempotency -Key per client	408/5xx,	exponential (100ms → 200ms → 400ms)	30s (p95 ≤ 30s NFR)	20s
POST /v1/documents /{id}/files	no	yes	Idempotency -Key per file upload	5xx	exponential	30s	25s
POST /v1/collections /{id}/submit	no	yes	Idempotency -Key unique per collection	5xx	linear/exponen- tial	10s	8s
POST /v1/collections /{id}/cancel	no	yes (If- Match)	If-Match (ETag)	5xx	linear (200ms)	5s	4s
DELETE /v1/files/{id}	no	yes	—	5xx	quick retries	5s	4s

Если клиент повторил запрос с тем же Idempotency-Key и тело запроса эквивалентно (проверка сигнатуры/хеша), сервер возвращает исходный

результат (тот же resource_id/код). Если тело отличается — вернуть 409 Conflict.

5.4 Политика версионирования

Стратегия версий: URL версионирование: /v{major}/... — простая и явная стратегия.

Совместимые (backward-compatible) изменения (minor/patch):

- Добавление новых необязательных полей в response.
- Добавление новых enum-значений, если клиенты игнорируют незнакомые значения.
- Добавление новых endpoints.

Ломающие изменения (require major version):

- Удаление/переименование обязательных полей.
- Изменение типов существующих полей (string→object).
- Изменение семантики статусов (например, status meaning change).
- Изменения формата ошибок.

Deprecation process:

1. Внести новую версию (v2). Поддерживать v1 минимум 90 дней (или SLA согласованное).
2. Пометить Deprecation заголовком в ответах: Deprecation: true, Sunset: <date> и публиковать changelog.
3. Нотификация клиентов (email, webhook, changelog).
4. После Sunset — отключение.

Матрица совместимости представлена в таблице 29.

Таблица 29 – Матрица совместимости

Client \ Server	v1	v2
v1	Полная	Отсутствие совместимости
v2	Только на чтение	Полная

5.5 Аутентификация/авторизация/безопасность

Межсервисная авторизация: Bearer JWT.

Клиентская авторизация: Bearer JWT (SSO).

Rate-limit: 100 req/min. Return 429 + Retry-After.

ETag / If-Match для конкурентных обновлений.

Валидация загружаемых файлов: проверка расширения, mime-type, max size (например 50MB), сканирование на вирусы, проверка содержимого ZIP.

Audit logs: 100% критичных операций (create collection, upload file, submit) — писать trace_id в ответе.

6 ПРАКТИЧЕСКАЯ РАБОТА №6

6.1 Карта доменных событий

Карта доменных событий представлена на рисунке 10.

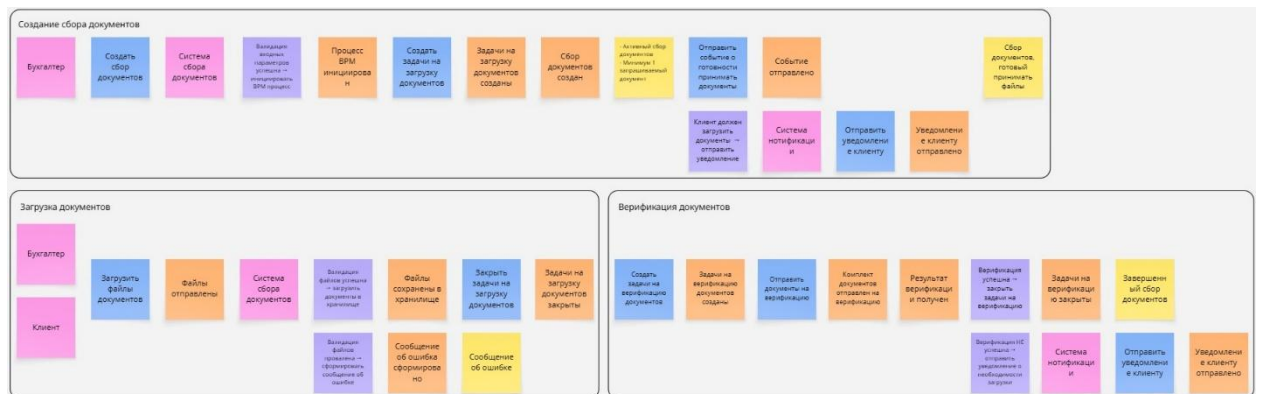


Рисунок 10 – Карта доменных событий

7 ПРАКТИЧЕСКАЯ РАБОТА №7

7.1 Сценарий и допущения

Сквозной сценарий (поток ценности): «Аутентификация (получение JWT) → Просмотр списка требуемых сборов документов (Join/Snapshot) → Загрузка файла (обмен состоянием, POST) → Подтверждение отправки на верификацию (POST /submit) → Выход (Logout или истечение токена)».

Допущения:

- Тип клиента: Web-приложение
- Сеть: WiFi.
- Целевой регион / RTT: РФ. Ожидаемый RTT p95 $\approx$ 150 мс.
- Допустимые потери: 0.
- Лимит участников: 1:1 (Клиент $\leftrightarrow$ Сервер).

7.2 Диаграмма последовательности

Диаграмма последовательности представлена на рисунке 11.

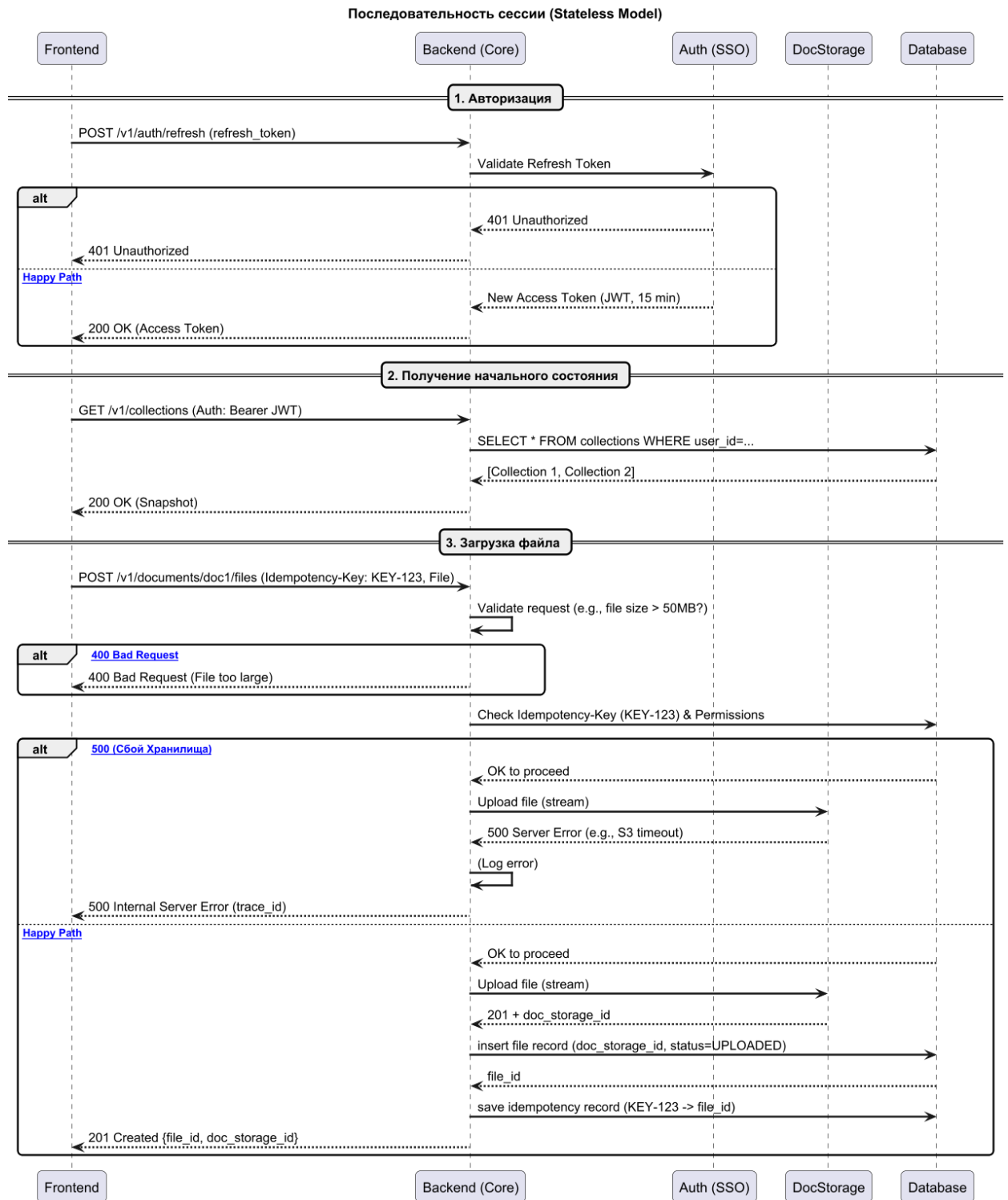


Рисунок 11 – Диаграмма последовательности

7.3 Диаграммы состояний

State-диаграммы представлены на рисунках 12-13.

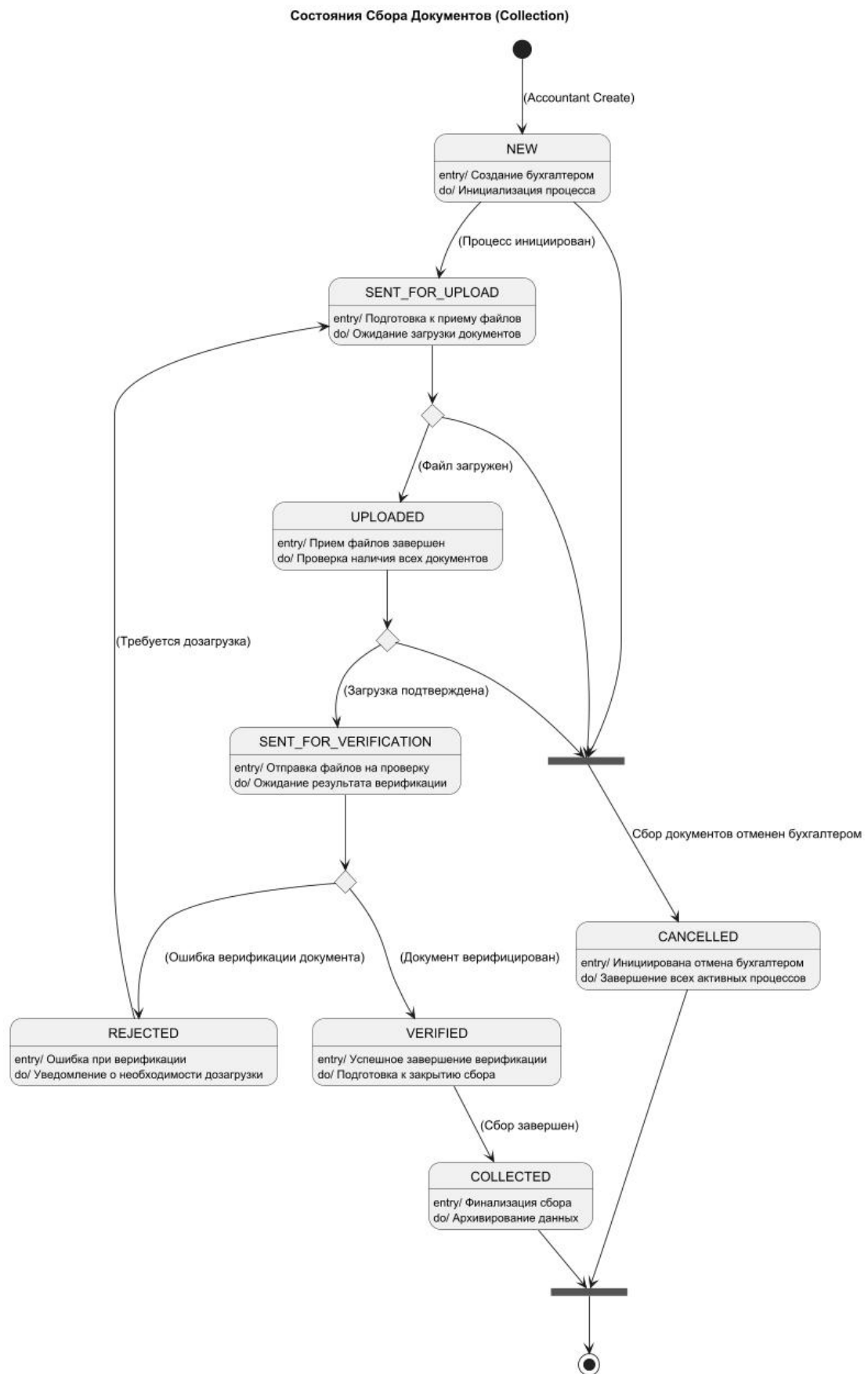


Рисунок 12 – State-диаграмма сбора документов

Состояния Запрашиваемого Документа (CollectionDocument)

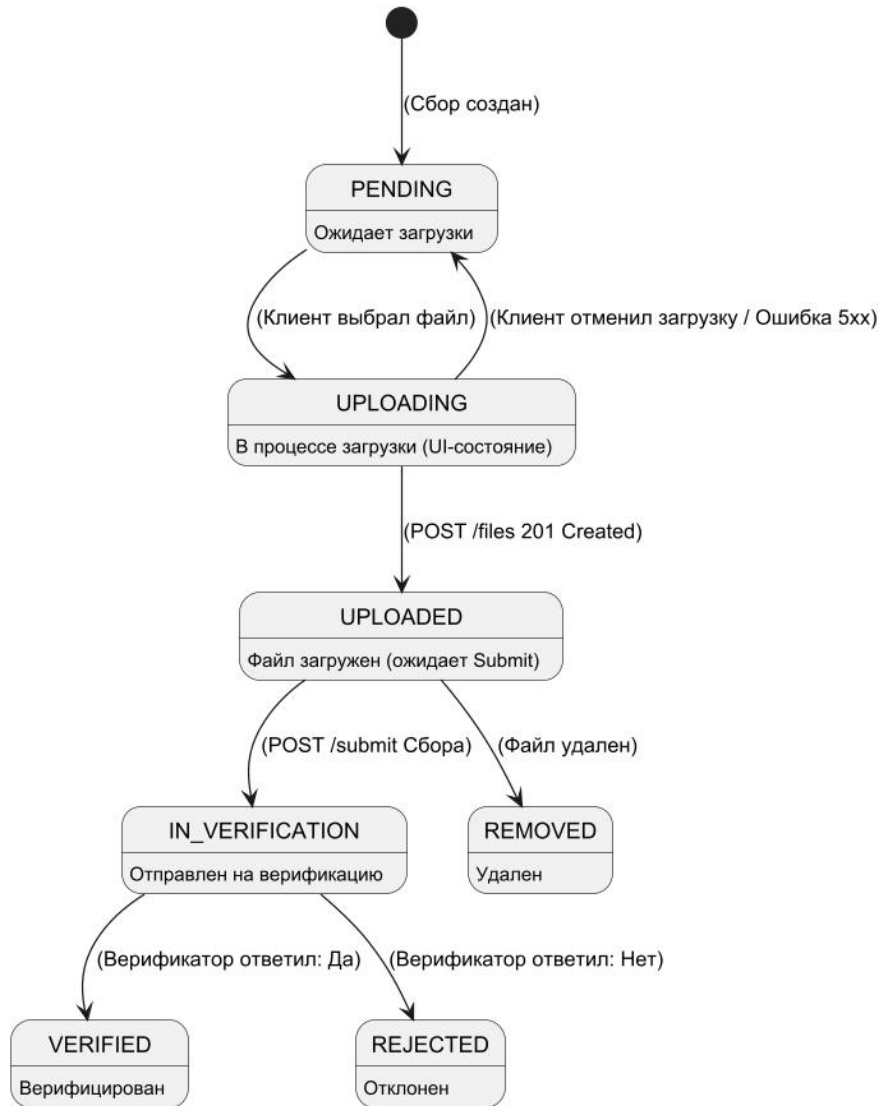


Рисунок 13 – State-диаграмма документа

7.4 Модель репликации состояния

Используется гибридный подход:

1. Снапшот (Snapshot): При входе в приложение (Join) клиент получает полный снимок состояния (GET /v1/collections) — список всех назначенных ему сборов и их текущие статусы.

2. Команды (Commands): Клиент отправляет на сервер *команды*, которые меняют состояние (POST /v1/documents/{id}/files или POST /v1/collections/{id}/submit).

3. Дельты (Polling): Клиент не использует потоковую репликацию (WebSockets). Для экономии батареи, он периодически (например, каждые 30 секунд или при Pull-to-Refresh) опрашивает (GET /v1/collections/{id}) сервер

для получения "дельт" — изменившихся статусов (например, status: IN_VERIFICATION -> status: VERIFIED).

Все команды, снапшоты и поллинг идут по HTTPS (TCP). Потеря команды (POST /submit) недопустима.

Матрица «Сущность → Канал → Политика» представлена в таблице 30.

Таблица 30 – Матрица «Сущность → Канал → Политика»

Сущность (Что реплицируется)	Канал	Политика (Частота, Надёжность)
Команда: Загрузка Файла	Надёжный (HTTPS POST)	По событию (действие клиента). Ретраи: Да, с Idempotency-Key.
Команда: Submit	Надёжный (HTTPS POST)	По событию (действие клиента). Ретраи: Да, с Idempotency-Key.
Состояние: Статусы сборов	Надёжный (HTTPS GET)	Pull (Polling). Частота: 1 раз в 30 сек (когда приложение активно) + Pull-to-Refresh.

7.5 Окна лаг-компенсации и сглаживания

В нашей ИС нет "лаг-компенсации" в игровом смысле (предсказание движения). Вместо этого мы используем предсказание UI для сглаживания сетевых задержек (RTT 150ms).

Буфер интерполяции (UI Smoothing):

Сценарий: Клиент нажимает "Загрузить файл".

Предсказание: UI немедленно (0 мс) переводит этот документ в состояние "Загружается..." (локальное предсказание) и показывает спиннер.

Реальность: Запрос уходит на сервер (RTT 150мс), сервер обрабатывает (NFR p95 до 5 сек).

Окно реконсиляции (Reconciliation):

Это окно — 1 последняя команда ("Upload").

Если (через 5 сек) приходит ответ 201 Created (Authoritative state): UI меняет "Загружается..." на "Загружено" (сверка прошла успешно).

Если (через 5 сек) приходит ответ 500 Server Error: UI меняет "Загружается..." на "Ошибка загрузки" (сверка провалилась) и разблокирует кнопку "Повторить".

7.6 Таблица параметров обновления

Таблица параметров обновления представлена в таблице 31.

Таблица 31 – Таблица параметров обновления

Категория	Параметр	Значение	Обоснование
Частоты	Частота снапшотов (Join)	1 раз / сессию	При входе получаем полный список сборов.
	Частота дельт (Polling)	0.033 Гц (1 раз / 30 сек)	Компромисс между актуальностью статусов и экономией батареи мобильного клиента.
Сеть	Heartbeat (мс)	N/A	Stateless-модель
	Timeout-порог (мс)	15 минут	JWT Access Token TTL. Это "таймаут" сессии.
	Max Packet (Upload)	50 МБ	NFR
Надёжность	Политика ретраев	Idempotency-Key (для POST) / Exponential Backoff (для GET)	
	Max Retries (для GET)	3	
Качество (SLI)	p95 Latency (API GET)	$\leq 1\,000$ мс	NFR
	p95 Latency (API POST File)	$\leq 5\,000$ мс	NFR
	Допустимая потеря пакетов	0%	Допущение

7.7 Связь с профилем

Надёжность: Все данные (файлы, статусы) критически надёжны. Нет "ненадёжных" (UDP) каналов, как в играх. Все идет по HTTPS (TCP).

Асинхронность: "Реальное время" достигается асинхронно. Когда клиент отправляет команду POST /submit, Backend (Core) немедленно отвечает

202 Accepted, а *внутри* бэкенда эта команда попадает в Брокер (BPM, RabbitMQ, как в Практике 1). Клиент *не* ждет верификации, он узнает о ее результате через поллинг.

Stateless: Отсутствие Heartbeat и использование JWT-токенов — ключевое решение для масштабируемости и экономии батареи мобильного клиента.

7.8 Наблюдаемость

Как мы поймем, что наша *stateless*-модель работает:

Дашборды

1. Auth Health: График 401 Unauthorized. Это наш главный индикатор "проблем с сессией". Если 401 много, значит у клиентов проблемы с обновлением (Refresh) токенов.

2. API Reliability:

- Счетчики (Rate) ошибок 5xx (Сбои Core, DocStorage).
- Счетчики (Rate) ошибок 400 (Клиент шлет невалидные данные).
- Счетчики (Rate) ошибок 409 Conflict (Ошибка идемпотентности).

3. Idempotency: % запросов POST, которые были повторными (показатель того, как часто ретраи в 5% сети спасают клиента).

Тревоги (Alerts)

- alert: High_401_Rate (Если 401 > 5% от запросов) → Проблема с SSO или атака. (Замена "Heartbeat_Drop").

- alert: High_5xx_Rate (Если p99 Latency POST /files > 5 сек) → Деградация DocStorage.

Тестовые сценарии (на бумаге)

1. Сценарий: "Потеря Wifi" (Loss 100% на 20 сек)

- *Действие*: Клиент в приложении. Проблемы на стороне провайдера.
- *Ожидание*: Ничего не происходит. "Сессия" не теряется, так как ее нет на сервере. Следующий API-запрос (e.g., Pull-to-Refresh) просто работает, так как JWT-токен (15 мин) еще валиден.

2. Сценарий: "Двойной Submit" (Retry)
 - *Действие*: Клиент жмет "Submit" (POST ... Idempotency-Key: KEY-SUBMIT). Запрос уходит, но ответ от сервера теряется (5% loss).
 - *Ожидание*: Клиентский SDK (не видя 202 Accepted) выполняет Retry с тем же Id...Key: KEY-SUBMIT.
 - *SLO*: Сервер видит ключ, *не* запускает вторую верификацию, а возвращает кешированный ответ 202 Accepted (как на Sequence-диаграмме). Модель устойчива.
3. Сценарий: "Протухший токен"
 - *Действие*: Клиент открыл приложение спустя 1 час (JWT 15 мин истек).
 - *Ожидание*: Первый запрос (e.g., GET /collections) возвращает 401. Клиентский SDK *автоматически* использует Refresh Token, получает новый JWT и *прозрачно* повторяет GET /collections. Пользователь видит только чуть более долгую загрузку (1 RTT + RTT Auth).

8 ПРАКТИЧЕСКАЯ РАБОТА №8

8.1 Эволюция контракта

Единица эволюции – Сбор документов (Collection).

- API: POST /v1/collections (Создание сбора).
- Схема/Таблица: collections.

Контекст:

- Доля старых клиентов (C1): 70%. (Мы должны поддерживать их долго).
- Требования: Миграция данных без даунтайма; поддержка старых клиентов (C1) на новых серверах (S2) в режиме обратной совместимости.

Паспорт контракта v1

API (Запрос POST /v1/collections v1):

- company_id (uuid, Обязательно)
- counterparty_id (uuid, Опционально)
- type (string, Обязательно, Enum)
- documents (array, Обязательно)
- pushing (string, Опционально, Enum: EMAIL, SMS, PUSH)
- expired_date (string, Опционально, RFC3339)

Схема (Таблица collections v1):

- id (uuid, PK)
- company_id (varchar, R)
- status (varchar, R)
- pushing (varchar, R, Enum)
- verification_template (varchar, R)
- expired_date (timestampz, R)

Смысловые инварианты (v1):

- Стратегия уведомления (pushing) — это простое строковое перечисление (EMAIL, SMS, PUSH).
- Сервер сам определяет verification_template

Проектирование v2 (Эволюция)

Причина изменений:

1. (Аддитивная) Бизнес хочет, чтобы бухгалтеры могли оставлять внутренние заметки к сбору, которые не видит клиент.
2. (Ломающая) Бизнес хочет, чтобы стратегии уведомлений (pushing) были не просто ENUM, а сложными, настраиваемыми сущностями (например, "EMAIL, если нет ответа 3 дня, то SMS").

Изменение 1 (Аддитивное/Совместимое):

- API: Добавляем internal_memo (string, Опционально) в запрос POST /v2/collections.
- DB: Добавляем столбец internal_memo (text, nullable) в таблицу collections.

Изменение 2 (Ломающее):

- API: Поле pushing (string) в POST /v2/collections заменяется на pushing_strategy_id (uuid, Опционально).
- DB: Столбец pushing (varchar) заменяется на pushing_strategy_id (uuid, FK, nullable).

8.2 ER-diff

Различия в контрактах версий API представлены в таблице 32.

Таблица 32 – Различия контрактов версий

Поле	v1 (POST /v1/collections)	v2 (POST /v2/collections)	Тип изменения
company_id	uuid (Обязательно)	uuid (Обязательно)	untouched
counterparty_id	uuid (Опционально)	uuid (Опционально)	untouched
type	string (Обязательно, Enum)	string (Обязательно, Enum)	untouched
documents	array (Обязательно)	array (Обязательно)	untouched
expired_date	string (Опционально)	string (Опционально)	untouched
pushing	string (Опционально, Enum)	Нет	Ломающее
pushing_strategy_id	(Отсутствует)	uuid (Опционально)	Ломающее (Замена pushing)
internal_memo	(Отсутствует)	string (Опционально)	Аддитивное

ER-diff диаграмма представлена на рисунке 14.

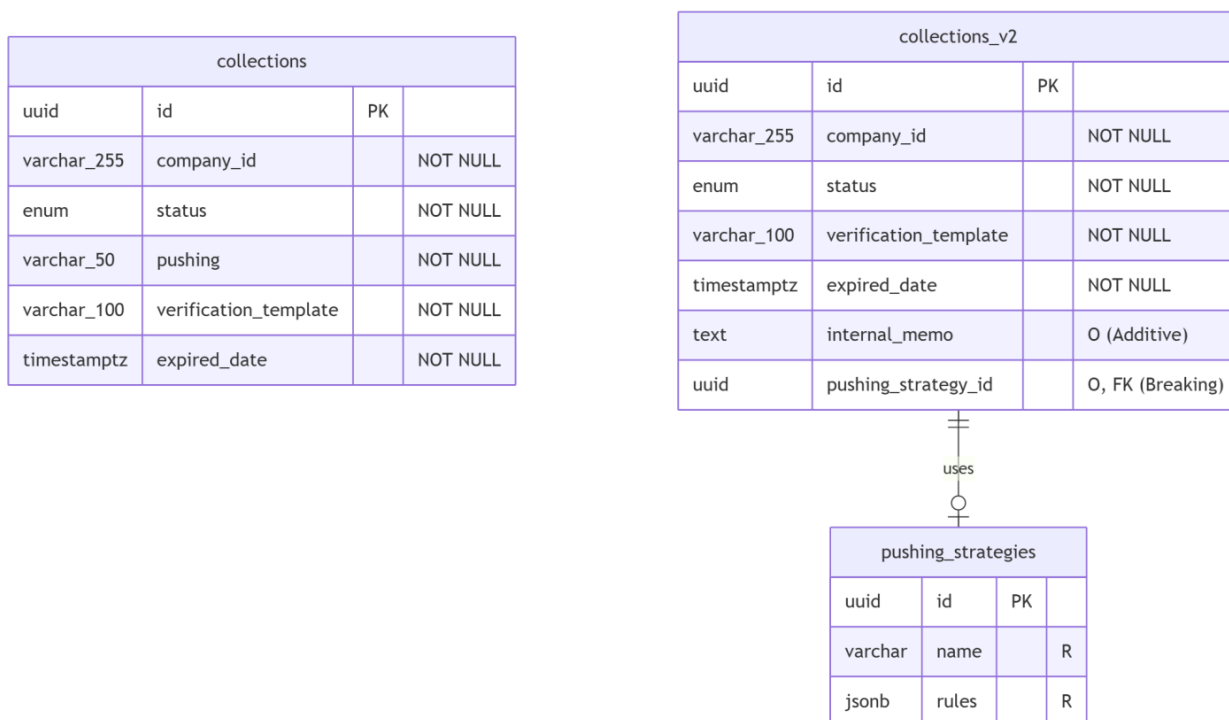


Рисунок 14 – ER-diff диаграмма

8.3 Матрица совместимости

C1: Старый клиент, использует /v1 , отправляет pushing: "EMAIL".

C2: Новый клиент, использует /v2, отправляет pushing_strategy_id: "uuid-...".

S1: Старый сервер, знает только о /v1.

S2: Новый сервер, поддерживает и /v1 (для C1) и /v2 (для C2).

Матрица совместимости представлена в таблице 33.

Таблица 33 – Матрица совместимости

Клиент \ Сервер	S1 (Только /v1)	S2 (Поддерживает /v1 и /v2)
C1	C1 шлет pushing: "EMAIL". S1 сохраняет "EMAIL".	C1 шлет pushing: "EMAIL" на /v1. S2 на эндпоинте /v1 преобразует "EMAIL" в pushing_strategy_id: "uuid-email" и сохраняет. internal_memo = NULL.
C2	S1 не имеет эндпоинта /v2. Сервер отвечает 404 Not Found.	C2 шлет pushing_strategy_id и internal_memo на /v2. S2 всё сохраняет.

C1	S1 не имеет эндпоинта /v2.	C1 шлет pushing: "EMAIL" на /v2. S2 (v2) ожидал pushing_strategy_id. Сервер отвечает 422 Unprocessable Entity.
----	----------------------------	--

8.4 Правила Negotiation и Деградации

Negotiation (Согласование): Основное согласование происходит через версионирование URL (/v1 или /v2). Клиенты C2 также должны отправлять заголовок X-Client-Version: 2 для метрик.

Fallback (Деградация): Режим Degraded (Fallback) (ячейка C1 -> S2) — самый важный.

- Правило: когда C1 отправляет pushing: "EMAIL" на эндпоинт /v1 сервера S2, S2 *обязан* это принять.
- Действие: S2 смотрит на свой pushing_strategy_mapping ('EMAIL' -> 'uuid-email') и подменяет данные.
- Почему это лучше ошибки: Мы сохраняем обратную совместимость. Клиент v1 не знает о v2, но его *намерение* (отправить email) выполняется.

8.5 Политика Депрекейта

Мы используем модель "Анонс -> Двойная работа -> Отключение".

1. Announce (Анонс) (День 0):

- Запускаем S2 с поддержкой /v2.
- Сервер S2 на запросы /v1 начинает отдавать заголовки: Deprecation: true и Sunset: <Today+90 days>.

2. Dual-run (Двойная работа) (Дни 0-120):

- Поддерживаем оба API (/v1 и /v2).
- Условие: Мониторим дашборд: "Доля трафика на /v1".
- План отката: Если на /v2 всплеск ошибок 5xx, откатываем S2 на S1.

3. Stop-write-old (Отключение записи) (День 120):

- Условие: Если трафик /v1 < 2%.
- Эндпоинты POST и DELETE на /v1 начинают возвращать 410 Gone. (GET еще работает).

4. Stop-read-old (Полное отключение) (День 180):

- Отключаем /v1 (возвращает 410 Gone на все).

8.6 Миграция данных

Миграция pushing (varchar) на pushing_strategy_id (uuid) без даунтайма:

1. Expand (Deploy 1):

- Добавляем таблицу pushing_strategies (с 'uuid-email' и т.д.).
- Добавляем столбец collections.pushing_strategy_id (nullable).
- Выкатываем S2-код, который при *записи* в /v1 (получая pushing: "EMAIL") записывает *в оба* столбца: pushing = "EMAIL" и pushing_strategy_id = "uuid-email".

2. Migrate (Async):

- Запускаем скрипт, который в фоне обновляет все старые записи: UPDATE collections SET pushing_strategy_id = ... WHERE pushing IS NOT NULL AND pushing_strategy_id IS NULL.

3. Contract (Deploy 2):

- Выкатываем S2-код, который при *чтении* из /v1 *перестает* смотреть на столбец pushing и читает только из pushing_strategy_id (конвертируя "uuid-email" обратно в "EMAIL" для ответа C1).

4. Contract (Deploy 3 - после Депрекейта):

- Удаляем столбец collections.pushing.

8.7 Наблюдаемость

Обязательные метки (Labels):

- api_version (из URL: v1 или v2).
- client_version (из заголовка X-Client-Version).

Сигналы деградации (Alerts):

- Alert 1: $\text{rate}(422_errors\{api_version="v2"\}) > 0$ — Сигнал: C1 пытается использовать v2.
- Alert 2: $\text{rate}(traffic\{api_version="v1"\})$ — Дашборд: Мониторинг прогресса депрекейта.

• Alert 3: rate(5xx_errors{api_version="v2"}) > rate(5xx_errors{api_version="v1"}) — Сигнал: v2 менее стабилен, чем v1 (нужен откат).

8.8 Наблюдаемость

1. Сценарий: C1 -> S1 (/v1)

- Запрос: POST /v1/collections, {"pushing": "EMAIL"}
- Ожидание: 201 Created. В БД pushing = "EMAIL". (Штатная работа v1).

2. Сценарий: C2 -> S2 (/v2)

- Запрос: POST /v2/collections, {"pushing_strategy_id": "uuid-email", "internal_memo": "note"}
- Ожидание: 201 Created. В БД pushing_strategy_id = "uuid-email", internal_memo = "note". (Штатная работа v2).

3. Сценарий: C1 -> S2 (/v1) (Деградация)

- Запрос: POST /v1/collections, {"pushing": "EMAIL"}
- Ожидание: 201 Created. S2 преобразовал данные. В БД pushing_strategy_id = "uuid-email", internal_memo = NULL. (Обратная совместимость S2 работает).

4. Сценарий: C2 -> S1 (/v1) (Деградация)

- Запрос: C2 (умный клиент) видит, что S1 (v1). Он сам выполняет преобразование. POST /v1/collections, {"pushing": "EMAIL"}.
- Ожидание: 201 Created. S1 сохранил pushing = "EMAIL". (Обратная совместимость C2 работает).

5. Сценарий: C1 -> S2 (/v2) (Блокировка)

- Запрос: C1 (старый клиент) ошибочно шлет на /v2. POST /v2/collections, {"pushing": "EMAIL"}.
- Ожидание: 422 Unprocessable Entity. Сообщение: field: pushing_strategy_id, message: is required (или "pushing is unknown field"). (S2 корректно заблокировал невалидную схему v1).

6. Сценарий: C2 -> S1 (/v2) (Блокировка)
 - Запрос: C2 шлет на /v2.
 - Ожидание: 404 Not Found. (S1 не знает о v2).
7. Сценарий: Ретрай C1 -> S2 (/v1)
 - Запрос: C1 шлет POST /v1/collections с Idempotency-Key: K1 и pushing: "EMAIL". S2 отвечает 201. C1 повторяет запрос (сеть моргнула).
 - Ожидание: 201 Created (из кэша идемпотентности). S2 *не* создает второй сбор. (Идемпотентность работает сквозь версии).

9 ПРАКТИЧЕСКАЯ РАБОТА №9

9.1 Сценарии

Система предназначена для автоматизации процесса запроса, сбора и верификации документов, необходимых для подтверждения бухгалтерских операций клиентов B2B. Ее бизнес-ценность заключается в снижении ручной работы бухгалтерии, ускорении обработки запросов и централизованном хранении документов.

Основной сценарий (Создание сбора): Бухгалтер, авторизованный в системе, инициирует новый "Сбор документов". Он указывает клиента (компанию), тип операции и перечень требуемых документов (например, "Счет-фактура за Март"). Система создает сбор в статусе "NEW" и автоматически отправляет уведомление клиенту о необходимости загрузить файлы.

Расширяющий сценарий (Загрузка и Верификация): Клиент получает уведомление, авторизуется в системе и видит список требуемых документов. Он загружает файлы (например, invoice.pdf). Система сохраняет файлы, связывает их со сбором и меняет статус на "UPLOADED". Затем система автоматически отправляет файлы во внешнюю "Систему верификации". После проверки статус обновляется на "VERIFIED" или "REJECTED" (если файл не прошел проверку), о чем клиент получает уведомление.

9.2 Измеримые нефункциональные требования

На основе Utility Tree, ключевыми NFR для системы являются:

1. Performance: p95 времени загрузки документа клиентом (отправка файла до 10 МБ) должен быть ≤ 5 секунд.
2. Availability: Доступность критичного клиентского пути (загрузка и скачивание документов) должна быть $\geq 99.9\%$.
3. Security: 100% всех клиентских и межсервисных операций должны требовать валидной авторизации (JWT для клиентов, MSA-токены для серверов).

9.3 Используемый паттерн и обоснование

Для управления доступом к микросервисам (Backend Core, Backend Accounting) мы выбрали паттерн API Gateway, реализованный через Nginx Ingress. Это решение было принято, поскольку оно решает три ключевые задачи.

Во-первых, шлюз централизует аутентификацию (проверку JWT), избавляя сервисы от дублирования этой логики. Во-вторых, он служит единой точкой входа, которая маршрутизирует запросы клиентов, скрывая внутреннюю топологию системы. В-третьих, Gateway упрощает управление сквозными NFR, такими как Rate Limiting и логирование.

Альтернатива в виде прямого доступа к сервисам была отклонена из-за проблем с безопасностью и дублированием логики. Паттерн BFF (Backend for Frontend) также был отклонен как избыточный, поскольку наши сценарии для веба и мобильных устройств идентичны.

9.4 Archimate диаграмма

Archimate диаграмма представлена на рисунке 15.

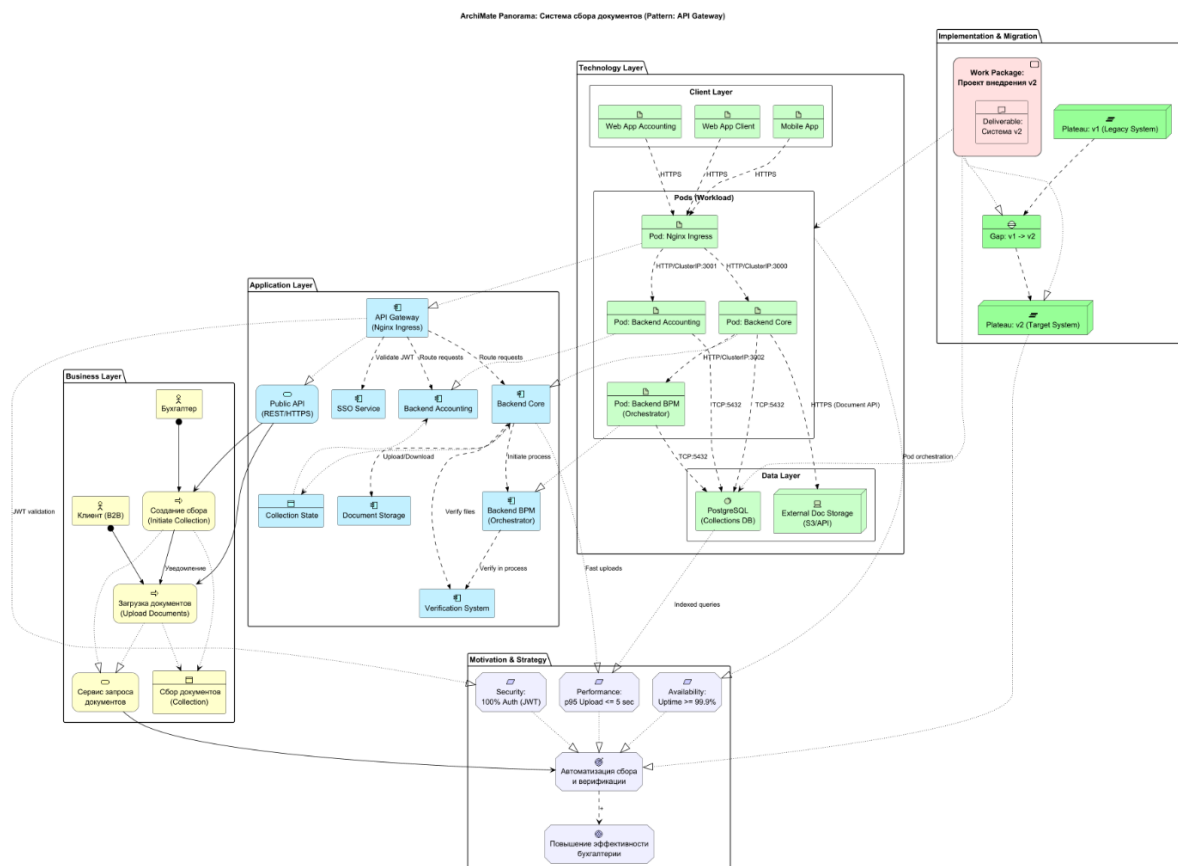


Рисунок 15 – Archimate диаграмма

10 ПРАКТИЧЕСКАЯ РАБОТА №10

10.1 Описание системы

Система предназначена для автоматизации запроса, сбора и верификации документов по бухгалтерским операциям для B2B-клиентов. Ключевые пользователи — Бухгалтер (инициирует сборы) и Клиент (загружает документы). Система интегрируется с внешними сервисами: SSO для авторизации, Хранилищем документов (DocStorage) для файлов, Системой верификации и Системой нотификаций.

10.2 C4-Context диаграмма

C4-context диаграмма представлена на рисунке 16.

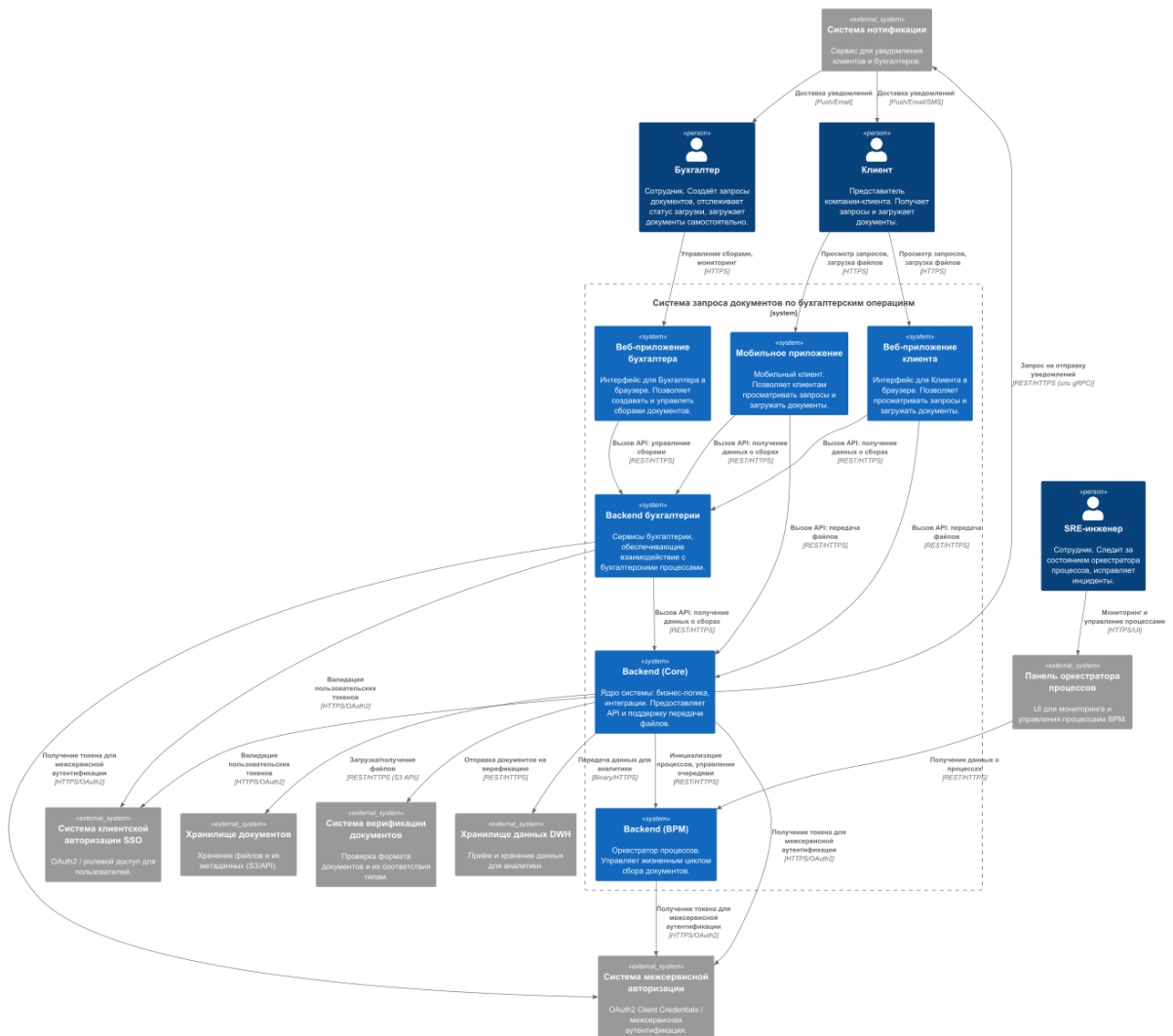


Рисунок 16 – C4-context диаграмма

10.3 C4-Container диаграмма

C4-Component диаграмма представлена на рисунке 17.

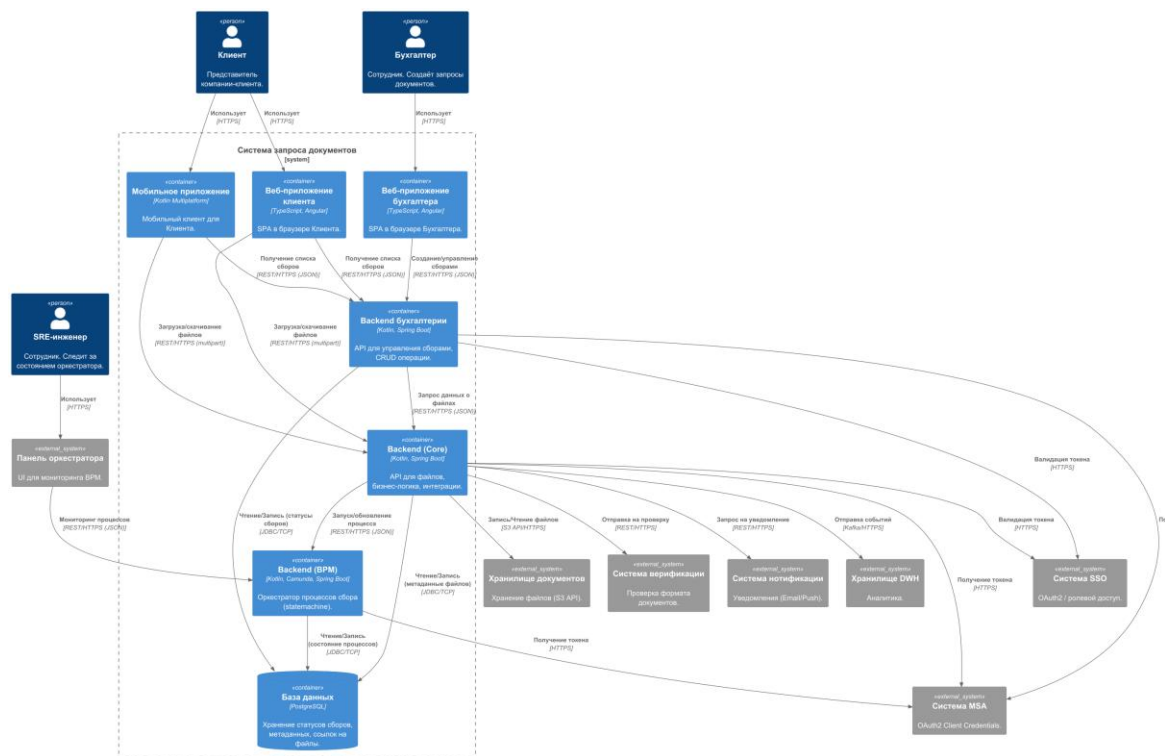


Рисунок 17 – C4-Container диаграмма

10.4 C4-Component диаграмма

C4-Component диаграмма представлена на рисунке 18.

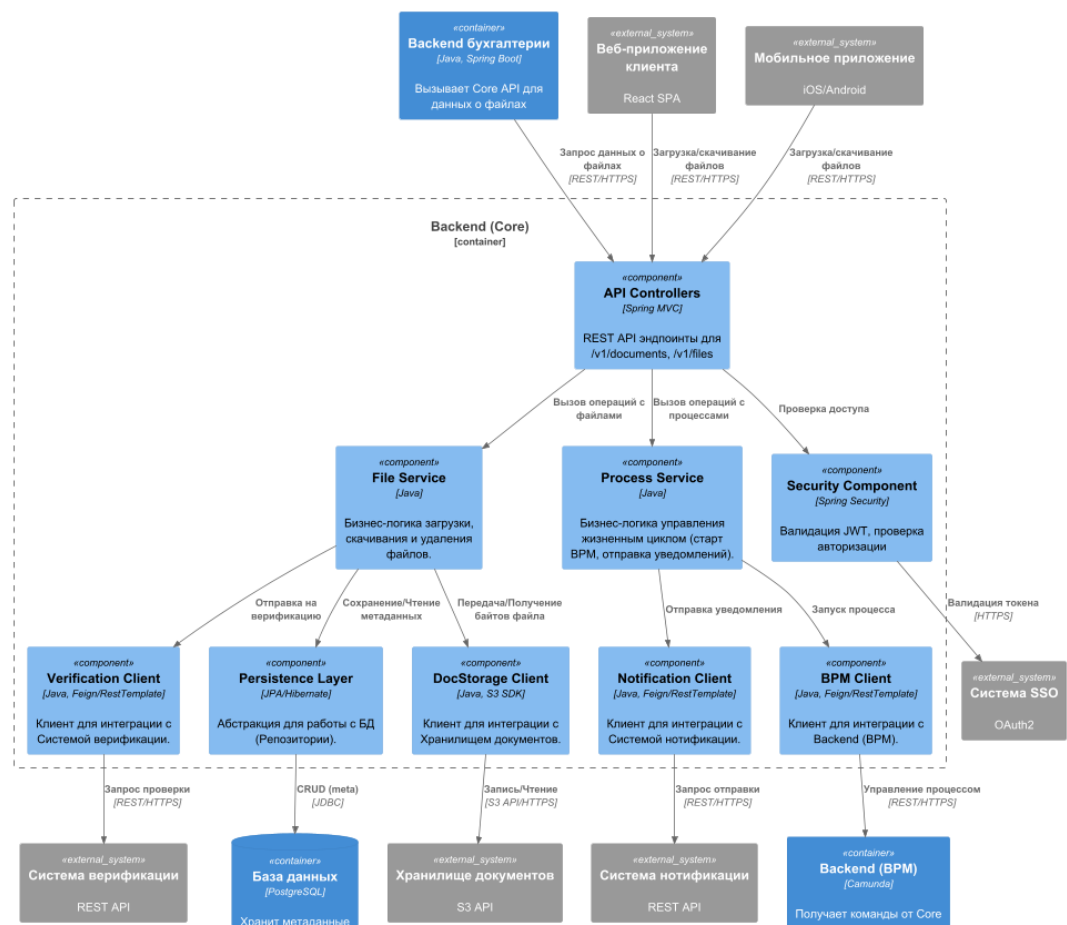


Рисунок 18 – C4-Component диаграмма

10.5 C4-Landscape диаграмма

C4-Landscape диаграмма представлена на рисунке 19.

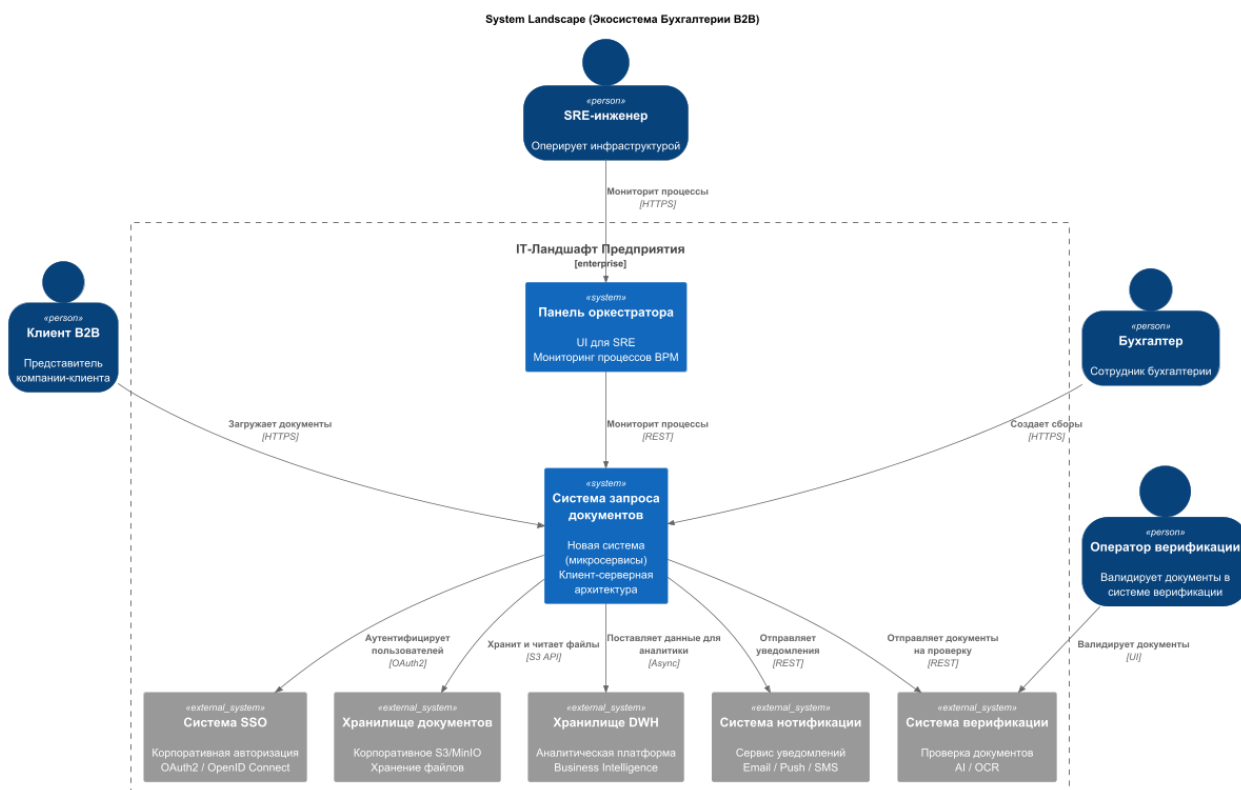


Рисунок 19 – C4-Landscape диаграмма